

SCALARIO LABS

SCALARIO

Architecture Technique Complète

Forge · Flow · Canvas · Calc · Sense · Vault · Mind · Shield

Outil interne de delivery → Plateforme SaaS
PME Africaines · BDUI · IA Conversationnelle · Multi-tenant

Mai 2026 — Version 9.0

PARTIE 1 — Vision & Positionnement

1. Vision — Scalario en 3 Piliers

Scalario est un moteur de livraison d'ERP sur mesure piloté par l'IA. En phase 1, c'est l'outil interne de Scalario Labs pour livrer des ERP à des clients 10x plus vite. En phase 3, c'est une plateforme SaaS accessible en self-service.

Pilier 1 — Backend-Driven UI (BDUI)

Le serveur génère dynamiquement la structure de l'interface selon le rôle, le contexte et les permissions.

Flutter reçoit un JSON schema et rend les composants sans aucun if métier dans le code.

Zéro mise à jour App Store pour changer l'UI — modifier la config suffit.

Pilier 2 — Configuration par IA Conversationnelle

Le client remplit un Business Profile à son rythme. L'IA analyse, génère un résumé technique et pose 3 à 7 questions de précision ciblées.

Un Demo Space multi-rôles permet de tester tous les workflows simultanément avant le go live.

Résultat : ERP opérationnel en moins d'une journée, pas en 6 mois.

Pilier 3 — RBAC + ABAC + Sécurité LLM

Chaque utilisateur a un rôle (RBAC) ET des attributs contextuels (ABAC : département, plafond, horaires).

Le LLM ne reçoit jamais de données brutes — tout est filtré par ABAC avant d'atteindre le modèle.

Row-Level Security PostgreSQL comme dernière ligne de défense.

2. Positionnement — SaaS ERP IA-Driven pour PME Africaines

Tu construis et héberges une plateforme SaaS. Tes clients ne gèrent aucun serveur — ils accèdent à leur ERP via une app universelle et une interface web. Tu es le Shopify des ERP pour les PME africaines.

Le modèle SaaS — tu héberges tout

1 infrastructure mutualisée — NestJS + PostgreSQL + MinIO + Redis + GPU cluster DeepSeek.

1 APK universelle publiée une fois sur les stores — le moteur BDUI vide.

Au login → NestJS identifie le tenant + rôle → pousse la config → Flutter rend l'UI.

Le client configure via Flutter Web (app.tonERP.com) — zéro installation serveur.

Le client utilise via l'app mobile/desktop — zéro gestion d'infrastructure.

Tu factures en abonnement mensuel selon le nombre d'utilisateurs.

2.1 Ce que le client fait vs ce que tu fais

Ce que TU fais (une fois)	Ce que LE CLIENT fait
Héberger toute l'infrastructure	Créer son compte sur app.tonERP.com
Publier l'APK sur les stores	Télécharger l'APK (une fois)
Maintenir les serveurs	Remplir le Business Profile
Gérer le cluster GPU DeepSeek	Inviter ses utilisateurs par email
Déployer les mises à jour config	Se connecter → son ERP apparaît
Garantir l'uptime et la sécurité	Payer l'abonnement mensuel

2.2 Pourquoi ce marché est non adressé

SAP / Oracle / Odoo	Ton ERP SaaS
Conçus pour DSI et grandes entreprises	Conçus pour PME africaines sans DSI
Config : consultant + 3-6 mois	Config : IA conversationnelle + 45 min
Prix : inaccessible pour PME	Abonnement mensuel accessible
Pas offline-first natif	Offline natif CRDT — zones sans réseau
Pas localisés (XOF, contextes locaux)	Conçu pour les marchés africains

Installation serveur requise	SaaS — zéro installation côté client
UI fixe imposée à tous	BDUI générée par rôle et contexte métier

Ton positionnement réel

Tu n'es pas en dessous de SAP — tu adresses un marché qu'ils ignorent.

SAP regarde les multinationales. Odoo regarde les PME avec DSI.

Toi tu cibles les centaines de millions de PME africaines qui gèrent encore avec Excel et cahiers.

Ce n'est pas leur concurrent direct — c'est leur successeur naturel sur ce marché.

3. Nomenclature Scalario

Chaque composant de la plateforme porte un nom Scalario. Cette nomenclature permet à l'équipe de parler clairement sans ambiguïté technique.

3.1 Vue d'ensemble — 7 engines

```

Scalario Labs (la boîte)
├─ Scalario (la plateforme + le client multi-plateforme)
│   └─ Android · iOS · Web · Windows · macOS · Linux
├─ Scalario Forge      ← Config Agent IA
├─ Scalario Stage      ← Demo Space multi-rôles
├─ Scalario Flow       ← ActionEngine (orchestrateur des 6 engines)
│   ├── Scalario Canvas ← ComponentRegistry + NavigationConfig + variantes
│   ├── Scalario Form   ← FormEngine (saisie + validation + calculs live)
│   ├── Scalario Calc   ← AlgoEngine (fonctions composables typées)
│   ├── Scalario Sense  ← CapabilityRegistry (hardware/système)
│   ├── Scalario Vault  ← DataSourceRegistry (Drift + PostgreSQL)
│   └─ Scalario Live    ← Realtime Engine (events serveur → app) ← NOUVEAU
├─ Scalario Mind       ← LLM DeepSeek V4 hébergé
│   ├── Scalario Memory ← Mem0 (mémoire utilisateur cross-sessions)
│   └─ Scalario Search  ← RAG hybride (vecteur + BM25 + reranking)
├─ Scalario Kit        ← Catalogue modules JSON
│   ├── Scalario Profile ← UX Profiles + variantes par métier
│   └─ Scalario Pipe    ← Pipelines ActionEngine configurés par l'IA
├─ Scalario Shield     ← Sécurité RBAC/ABAC/RLS
│   ├── Scalario Sync   ← CRDT offline
│   └─ Scalario Watch   ← Observabilité + Langfuse
└─ Phase 4 (vision long terme)
    └─ Scalario Network ← Réseau inter-tenants
        ├── Scalario Trade ← Transactions B2B cross-tenant
        └─ Scalario Market ← Marketplace sectorielle
  
```

3.2 Les 3 natures des composants

Nature	Composants	Pourquoi cette nature
Engines (7)	Flow, Canvas, Form, Calc, Sense, Vault, Live	Traient des données, produisent un résultat, ont un contrat JSON
Configs (pas des engines)	NavigationConfig, State/Error layer	Déclaratifs — évalués par Flutter, pas de logique métier propre
Catalogues	Kit, Profile, Pipe	Fichiers JSON statiques — pas de code

d'exécution

Pourquoi NavigationConfig n'est pas un engine

Un engine traite des données et produit un résultat (Canvas rend, Calc calcule, Vault lit).
 La navigation déclare juste : 'depuis cet écran, ces routes sont accessibles selon ce rôle'.
 C'est du JSON évalué par go_router Flutter — pas de logique métier propre.
 NavigationConfig est une section du JSON tenant_config, gérée par Scalario Canvas.

Pourquoi State/Error n'est pas un engine

La gestion d'état (loading, errors, retry, session expirée) est une couche technique Flutter.
 Elle est implémentée avec Riverpod — invisible pour l'IA et la config JSON.
 Ce n'est pas nommé publiquement car c'est de l'infrastructure Flutter standard.
 Documenté dans les guidelines développeur — pas dans la nomenclature Scalario.

3.3 Référence rapide complète

Nom Scalario	Rôle technique	Technologie
Scalario	Plateforme + client universel Flutter	Flutter (Android/iOS/Web/Windows/macOS/Linux)
Scalario Forge	Config Agent IA — Business Profile → ERPConfig	FastAPI + LangChain + Instructor + Pydantic
Scalario Stage	Demo Space — test multi- rôles avant go live	Flutter Web + sandbox NestJS
Scalario Flow	ActionEngine — orchestre les 7 engines via pipelines JSON	NestJS + Dart
Scalario Canvas	ComponentRegistry — widgets + variantes + NavigationConfig	Flutter + Material 3 + go_router
Scalario Form	FormEngine — saisie, validation, calculs temps réel	Flutter + Scalario Calc
Scalario Calc	AlgoEngine — fonctions atomiques composables, typage Zod/Dart	TypeScript + Dart (dual runtime)
Scalario Sense	CapabilityRegistry — scanner, GPS, print BT, NFC, signature, Mobile Money	Flutter platform channels + Wave/OM/MTN APIs
Scalario Vault	DataSourceRegistry — Drift offline + PostgreSQL online	Drift + TypeORM + pgvector + SQL catalogue
Scalario Live	Realtime Engine —	NestJS WebSocket + FCM (Android) +

	événements serveur → app (notifications, refresh)	APN (iOS) + Web Push
Scalario Mind	LLM principal — DeepSeek V4 hébergé sur cluster GPU	DeepSeek V4-Flash/Pro + FastAPI
Scalario Memory	Mémoire utilisateur cross-sessions	Mem0
Scalario Search	RAG hybride — vecteur + BM25 + RRF + reranking	pgvector + pg_trgm + Cohere Rerank
Scalario Kit	Catalogue modules JSON standards (commandes, stock, factures...)	JSON + catalog/modules/
Scalario Profile	UX Profiles + variantes autorisées par secteur métier	JSON + catalog/ux_profiles/
Scalario Pipe	Pipelines ActionEngine configurés par Scalario Forge	JSON + Scalario Flow
Scalario Shield	RBAC + ABAC + RLS — 5 couches de sécurité	CASL + Casbin + PostgreSQL RLS
Scalario Sync	Synchronisation offline CRDT avec Vector Clocks	Drift + CRDT + sync queue NestJS
Scalario Watch	Observabilité + traçage LLM + audit logs	Langfuse + NestJS Interceptors

4. Vision 3 Phases — Outil Interne → SaaS

Scalario ne commence pas comme un SaaS. Il commence comme l'outil interne de Scalario Labs pour livrer des ERP à des clients rapidement. La progression est délibérée.

Phase	Qui configure ?	Modèle économique
Phase 1 — Outil interne (maintenant → 18 mois)	Scalario Labs opère Scalario Forge pour les clients	Projets clients — revenus immédiats
Phase 2 — Semi self-service (18-30 mois)	Client remplit Business Profile, Labs supervise	Projets + premiers abonnements
Phase 3 — SaaS complet (30+ mois)	Client configure en full self-service	Abonnements mensuels scalables

4.1 Phase 1 — Outil interne de delivery

Ce que Phase 1 permet

Scalario Labs livre des ERP à des clients 10x plus vite qu'une agence classique.
 C'est Scalario Labs qui lance Scalario Forge pour chaque nouveau client.
 Le client reçoit un ERP fonctionnel en quelques jours — pas en 6 mois.
 Chaque client est un tenant dans l'infrastructure Scalario Labs.
 Les revenus sont des projets client — pas des abonnements.
 Objectif : affiner le moteur sur des vrais cas avant de scaler.

Ce que Phase 1 simplifie (pas besoin encore)

Pas de self-service onboarding — Labs crée les tenants manuellement.
 Pas de billing automatisé — facturation projet classique.
 Pas de portail client public — interface admin interne.
 Pas de support ticketing — relation directe client.
 Pas de landing page SaaS — Labs vend des projets.

4.2 Ce que Phase 1 construit pour Phase 3

Chaque client Phase 1 apporte :

- Des vrais patterns métier pour Scalario Kit
- Des corrections de Scalario Forge (hallucinations détectées)
- Des nouveaux UX Profiles pour Scalario Profile
- Des capacités manquantes pour Scalario Sense
- La preuve que le moteur fonctionne sur de vrais métiers

```
Après 10-15 clients Phase 1 :  
→ Scalario Forge est fiable sur 80% des cas  
→ Scalario Kit couvre les principaux secteurs  
→ L'infrastructure est battle-tested  
→ Phase 2 peut démarrer avec confiance
```

PARTIE 2 — Architecture Globale

5. Stack Technique — Décisions & Pourquoi

5.1 Décision principale : NestJS seul, pas Supabase

Pourquoi NestJS seul suffit

Supabase self-hosted = 8 services Docker (GoTrue, Realtime Elixir, Storage, Kong, Studio...) pour 4-8 GB RAM.

NestJS couvre déjà tout : Auth (Passport.js + JWT), Realtime (WebSocket Gateway), Storage (MinIO).

MinIO remplace Supabase Storage en 1 service léger S3-compatible.

Adminer remplace Supabase Studio pour visualiser la DB en dev.

Résultat : 5 services Docker au lieu de 12+, contrôle total, stack cohérente.

3.2 Les 5 services Docker

```
# docker-compose.yml – tout ce dont tu as besoin
services:
  nestjs:      # API principal – auth, BDUI, modules, workflows, engines
  fastapi:     # Microservice IA – RAG, streaming, Config Agent
  postgresql:  # Données + pgvector + RLS + schema-per-tenant
  redis:       # Cache config, sessions, rate limiting LLM
  minio:       # Storage fichiers – PDFs, ordonnances, images (S3-compatible)

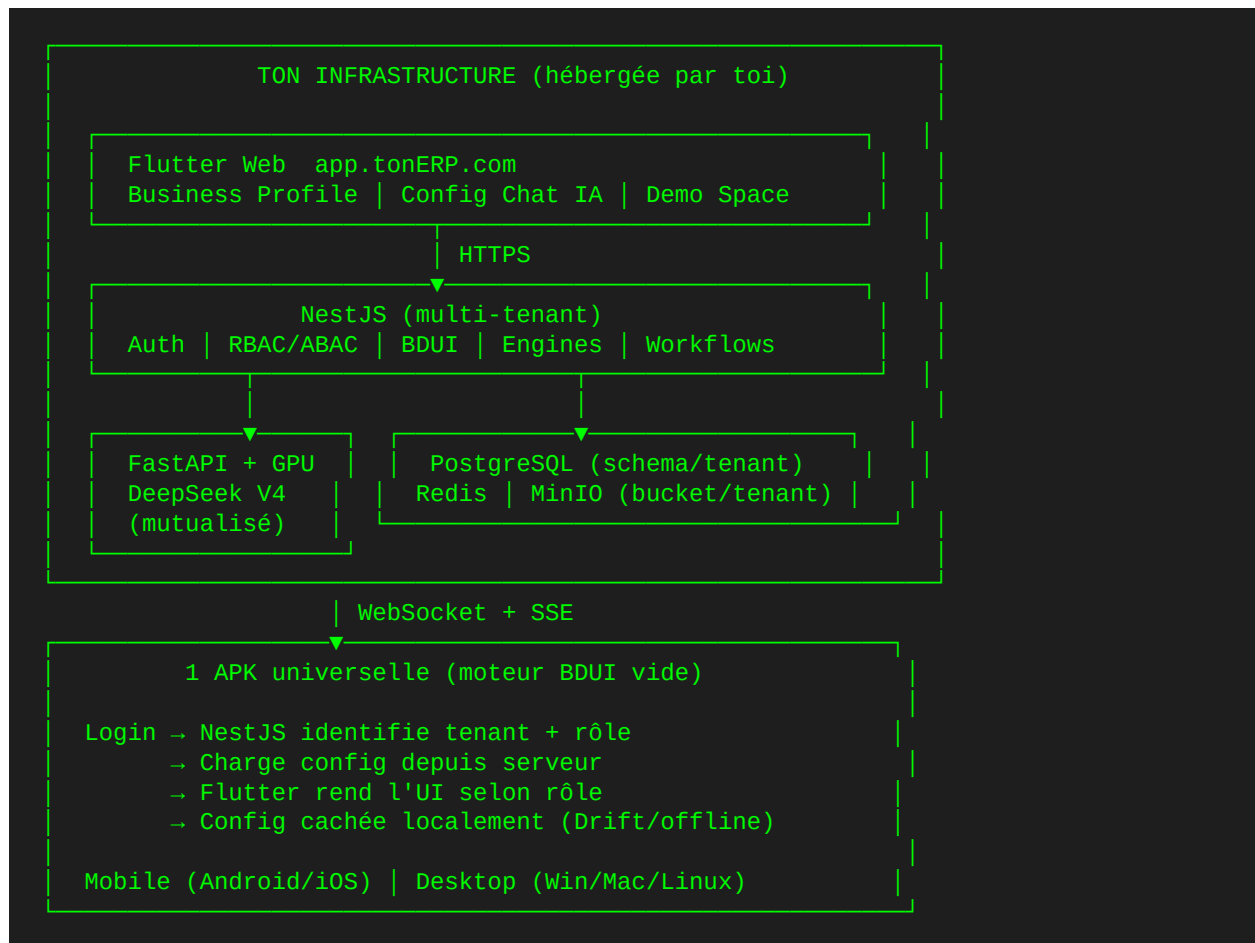
# Dev seulement :
adminer:      # Dashboard visuel DB – remplace Supabase Studio
```

3.3 Stack complète

Couche	Technologie	Rôle
App mobile	Flutter + Material 3 + Drift/Isar	BDUI renderer + offline-first + capabilities
App desktop	Flutter Desktop (Windows/macOS/Linux)	Managers, DG — écrans larges
App web (admin)	Flutter Web — même codebase	Config chat, preview BDUI, back-office
Design System	Material 3 + ThemeData custom	Tokens, composants, layouts — natif Flutter
Maquettage	Widgetbook (Flutter)	Storybook vivant — composants + états
API principal	NestJS	Auth, RBAC/ABAC, tous les engines
Auth	Passport.js + @nestjs/jwt	JWT, refresh tokens, OAuth2

Realtime	NestJS WebSocket Gateway	Streaming IA, notifications temps réel
Microservice IA	FastAPI + LangChain	RAG, Config Agent, streaming LLM
Base de données	PostgreSQL + pgvector	Données + RAG vectoriel + RLS
Cache	Redis	Sessions, config cache, rate limiting LLM
Storage	MinIO (S3-compatible)	PDFs, ordonnances, images — isolé par tenant
Mémoire IA	Mem0	Contexte long terme par utilisateur
Parsing docs	Docling (IBM)	PDF, Excel, Word → chunks indexables
RAG	LlamaIndex + pgvector	Hybrid search données métier
Permissions	CASL + Casbin	ABAC complexe dans NestJS
LLM principal	DeepSeek V4-Flash (hébergé)	Sur ton cluster GPU — zéro coût par requête
LLM raisonnement	DeepSeek V4-Pro (hébergé)	Config Agent, tâches complexes
LLM fallback sensible	Claude API (Sonnet)	Données critiques — paie, finance, santé
Cluster GPU	2-4x RTX 4090 ou A100	DeepSeek V4 distillé — mutualisé tous tenants

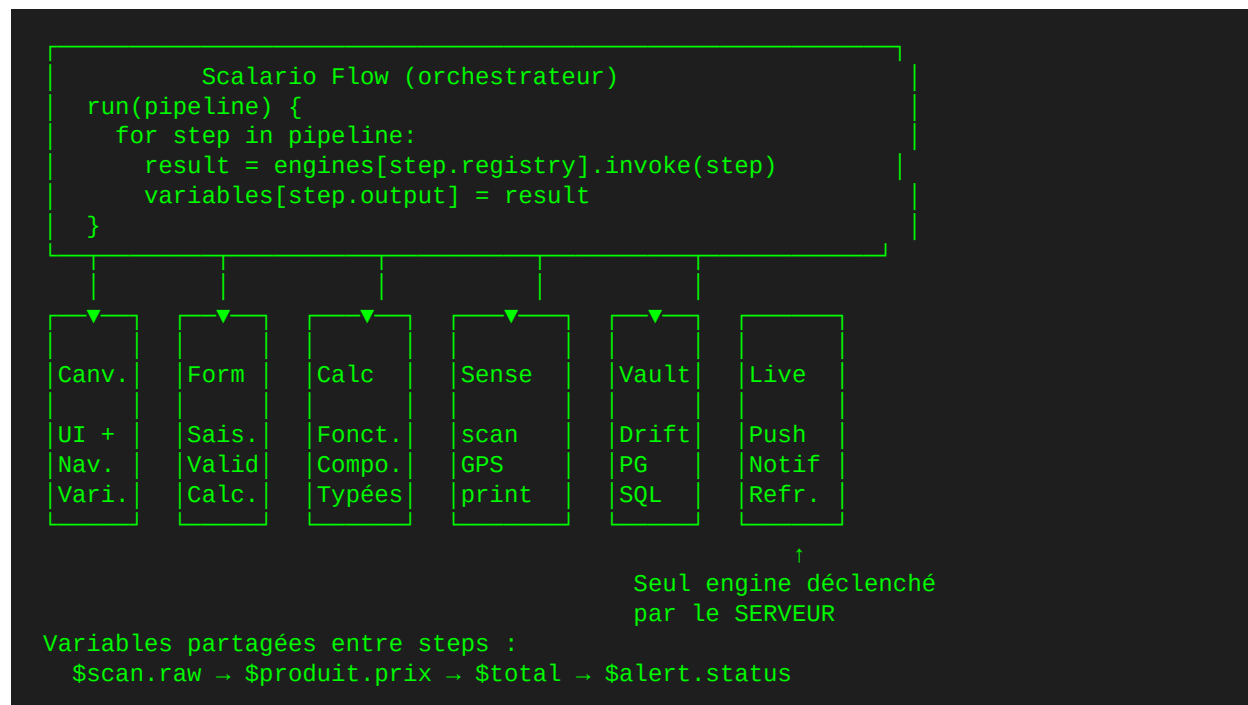
6. Architecture — Vue d'Ensemble



PARTIE 3 — Les 6 Engines — Le Moteur Central

7. Vue d'Ensemble — Les 7 Engines

Scalario Flow orchestre 6 engines spécialisés + lui-même = 7 au total. Ils communiquent via des variables partagées dans chaque pipeline. Scalario Live est le seul engine déclenché par le serveur — les 6 autres sont déclenchés par l'utilisateur.



Engine Scalario	Responsabilité	Déclenché par
Scalario Canvas	UI depuis JSON config + navigation + variantes contextuelles	Flow (user action)
Scalario Form	Saisie, validation, calculs temps réel, capacités dans champs	Flow (user action)
Scalario Calc	Fonctions composables typées (somme, tva, si, filtre...)	Flow (user action)
Scalario Sense	Hardware/système (scan, GPS, print BT, NFC, signature...)	Flow (user action)
Scalario Vault	Accès données — Drift offline / PostgreSQL online + SQL catalogue	Flow (user action)
Scalario Live	Événements serveur → app (notifications, refresh, badges, alerts)	NestJS WebSocket

Scalario Flow	Orchestrateur — exécute les pipelines JSON étape par étape	User trigger / événement
---------------	--	--------------------------

Principe clé — tout est un pipeline JSON

Une action utilisateur (clic, scan, timer) déclenche un pipeline Scalario Flow.

Scalario Live est l'exception — il réagit aux événements venant du serveur NestJS.

Les variables circulent entre les steps via \$nom_variable.

L'IA configure les pipelines — elle assemble depuis les catalogues Scalario Kit et Scalario Pipe.

Ajouter une fonctionnalité = ajouter au catalogue, pas modifier le code des engines.

8. Scalario Canvas — ComponentRegistry & Variantes

Scalario Canvas est le registre fermé des composants UI. Chaque composant a des variantes — l'IA choisit le type ET la variante. La variante 'auto' laisse Flutter adapter selon le contexte (écran, rôle, densité de données).

8.1 Principe — 1 type, N variantes

Pourquoi les variantes et pas des types séparés

Sans variantes : KPICard, KPICardCompact, KPICardWithIcon, KPICardHero → le catalogue explose.

Avec variantes : KPICard + variant = 'compact' → le registre reste propre, 1 entrée par composant.

variant: 'auto' → Flutter choisit selon la taille d'écran, le nombre d'éléments, le rôle de l'utilisateur.

L'IA choisit parmi les variantes autorisées dans Scalario Profile — elle n'en invente pas.

8.2 Catalogue des composants et variantes

Composant	Variantes disponibles	variant: auto choisit selon
KPICard	default · compact · with-icon · hero · with-chart	Taille écran + nb KPIs + rôle
DataTable	default · compact · card-list · timeline	Mobile → card-list, desktop → default
ListTile	default · with-avatar · with-badge · dense	Densité données + type entité
AlertBanner	info · success · warning · danger · dismissible	Statut de la donnée source
ChartBar	default · stacked · horizontal · mini	Espace disponible + nb séries
ChartPie	default · donut · mini-legend	Nb catégories + espace
Button	primary · secondary · ghost · danger · icon-only	Contexte action + espace
FAB	default · extended · mini	Mobile → mini, desktop → extended
FormField	text · number · date · select · search · scan	Type de donnée déclaré
StatCard	default · trend-up · trend-down · flat	Variation vs période précédente
SyncStatusBar	syncing · synced · conflict ·	Widget permanent dans le layout — état

	offline	Scalario Sync
DocumentPreview	inline · card · fullscreen · thumbnail	Preview PDF/facture avant impression

SyncStatusBar — widget permanent lié à Scalario Sync

SyncStatusBar est toujours visible dans le layout — il reflète l'état de Scalario Sync en temps réel.

variant: syncing → spinner + 'Synchronisation...'

variant: synced → icône verte + 'À jour' (disparaît après 3s)

variant: conflict → icône orange + 'Conflits à résoudre' → tap → ConflictReviewScreen

variant: offline → icône grise + 'Hors ligne — données locales'

Mis à jour automatiquement par Scalario Live via WebSocket.

ConflictReviewScreen — résolution de conflits CRDT

Écran dédié quand Scalario Sync détecte un conflit Vector Clock non résolu automatiquement. Affiche les 2 versions : locale (offline) et serveur (online) côte à côte.

L'utilisateur choisit la version à garder — ou Scalario Mind propose une fusion intelligente.

Chaque résolution est auditée dans Scalario Watch avec le contexte du conflit.

Les conflits simples (champs différents, pas de chevauchement) sont résolus auto sans UI.

8.3 Le JSON config — type + variant

```
// Variante explicite — l'IA choisit
{
  "type": "KPICard",
  "variant": "with-icon",
  "props": {
    "label": "Ruptures stock",
    "icon": "warning",
    "value_source": "stock.ruptures",
    "status": "danger"
  }
}

// Variante auto — Flutter adapte au contexte
{
  "type": "KPICard",
  "variant": "auto",
  "props": { "label": "Ventes jour", "value_source": "ventes.today" }
}

// → Mobile avec 4 KPIs : variant compact
// → Desktop avec 1 KPI : variant hero
// → Rôle DG : variant with-chart
// → Rôle Vendeur : variant default
```

8.4 Structure Flutter — dispatch par variante

```
// ComponentRegistry — 1 entrée par composant
class ComponentRegistry {
  static Widget build(ComponentConfig config, BuildContext ctx) {
    return switch (config.type) {
      'KPICard'    => KPICard.fromConfig(config, ctx),
      'DataTable' => ERPDataTable.fromConfig(config, ctx),
      'AlertBanner'=> AlertBanner.fromConfig(config, ctx),
      'ChartBar'  => ChartBar.fromConfig(config, ctx),
      // 1 entrée par composant — pas d'explosion
      _           => UnknownComponent(config.type),
    };
  }
}

// KPICard — dispatch interne par variante
class KPICard extends StatelessWidget {
  static Widget fromConfig(ComponentConfig config, BuildContext ctx) {
    final variant = resolveVariant(config.variant, ctx);
    return switch (variant) {
      'compact'    => KPICardCompact(config.props),
      'with-icon'  => KPICardWithIcon(config.props),
      'with-chart' => KPICardWithChart(config.props),
      'hero'       => KPICardHero(config.props),
      _            => KPICardDefault(config.props),
    };
  }
}

// Résolution de 'auto' selon le contexte
static String resolveVariant(String variant, BuildContext ctx) {
  if (variant != 'auto') return variant;
  final screen = MediaQuery.of(ctx).size;
  final user   = ctx.read<UserContext>();
  final kpiCount = ctx.read<ScreenState>().kpiCount;
  if (screen.width < 600) return 'compact';
  if (user.role == 'DG') return 'with-chart';
  if (kpiCount == 1)     return 'hero';
  return 'default';
}
}
```

8.5 Scalario Profile — variantes autorisées par métier

```
// catalog/ux_profiles/pharmacie/components.json
{
  "KPICard": {
    "default_variant": "with-icon",
    "mobile_variant": "compact",
  }
}
```

```

    "desktop_variant": "hero",
    "allowed_variants": ["default", "compact", "with-icon", "hero"]
    // with-chart non autorisé en pharmacie (trop complexe)
  },
  "DataTable": {
    "default_variant": "default",
    "mobile_variant": "card-list",
    "desktop_variant": "default",
    "allowed_variants": ["default", "compact", "card-list"]
  }
}

// L'IA ne peut choisir que parmi allowed_variants
// Instructor rejette toute variante hors liste
// Scalario Forge utilise default_variant si non précisé

```

8.6 Widgetbook — chaque variante documentée

```

WidgetbookComponent(
  name: 'KPICard',
  useCases: [
    // Default
    WidgetbookUseCase(name: 'Default – normal',
      builder: (_) => KPICard(variant:'default', value:'142', label:'Commandes')),
    WidgetbookUseCase(name: 'Default – warning',
      builder: (_) => KPICard(variant:'default', status:'warning', value:'3')),
    // Compact (mobile)
    WidgetbookUseCase(name: 'Compact – mobile 4 KPIs',
      builder: (_) => KPICard(variant:'compact', value:'142', label:'Ventes')),
    // With-icon
    WidgetbookUseCase(name: 'With-icon – danger',
      builder: (_) => KPICard(variant:'with-icon', icon:'warning',
status:'danger')),
    // Hero
    WidgetbookUseCase(name: 'Hero – seul sur desktop',
      builder: (_) => KPICard(variant:'hero', value:'2 450 000', label:'CA
Jour')),
    // Auto – simuler les contextes
    WidgetbookUseCase(name: 'Auto – mobile DG',
      builder: (ctx) => KPICard(variant:'auto', ...)),
  ]
)

```

8.7 Design System — 3 couches

Couche 1 — Design Tokens (jamais touchés par l'IA)

Couleurs : primary, success, danger, surface — définis une fois, gravés dans le marbre.

Espacement : grille de 4px — [4, 8, 12, 16, 24, 32, 48, 64].
Typographie : h1, h2, body, caption — tailles et graisses fixes.
L'IA ne mentionne jamais une couleur, une taille ou une police dans sa config.

Couche 2 — Material 3 + ThemeData custom

Material 3 est natif Flutter — zéro dépendance externe, tous les primitifs inclus.
Tu surcharges ThemeData avec tes design tokens — les variantes héritent automatiquement.
Chaque variante d'un composant utilise les mêmes tokens — cohérence garantie.
Flutter Web + Mobile + Desktop = même Design System, preview identique partout.

Couche 3 — Règles de layout (grammaire de l'UI)

Dashboard → max 4 KPICards en haut, variant auto recommandé, jamais 3 colonnes mobile.
Liste → SectionHeader obligatoire, DataTable.default desktop / DataTable.card-list mobile.
Formulaire → max 5 champs par section, bouton submit toujours en bas.
Un layout invalide est rejeté par NestJS avant sauvegarde.

8b. Internationalisation — i18n

Scalario cible les PME africaines. Un ERP sans i18n dès le départ crée une dette technique majeure. Les ARB files et le lint no_hardcoded_strings sont non négociables dès Phase 1.

8b.1 Langues cibles

Langue	Priorité	Marché
Français (FR)	Phase 1 — obligatoire	Afrique francophone — Burkina, Côte d'Ivoire, Sénégal, Mali...
Anglais (EN)	Phase 1 — obligatoire	Afrique anglophone — Ghana, Nigeria, Kenya...
Bambara (BM)	Phase 2	Mali, Burkina Faso — langue véhiculaire majeure
Wolof (WO)	Phase 2	Sénégal — 80% de la population
Haoussa (HA)	Phase 2	Nigeria, Niger, Ghana — 50M+ locuteurs
Dioula (DY)	Phase 2	Burkina, Côte d'Ivoire, Mali
Arabe (AR)	Phase 3	Maghreb + Sahel

8b.2 Structure ARB Files

```
lib/
└─ l10n/
   ├── app_fr.arb    ← Français (référence)
   ├── app_en.arb    ← Anglais
   ├── app_bm.arb    ← Bambara (Phase 2)
   └─ app_wo.arb    ← Wolof (Phase 2)

// app_fr.arb – exemple
{
  "@@locale": "fr",
  "welcomeMessage": "Bienvenue sur Scalario",
  "@welcomeMessage": { "description": "Message accueil login" },
  "commandesTitle": "Commandes",
  "validerCommande": "Valider la commande",
  "stockRupture": "Rupture de stock : {produit}",
  "@stockRupture": {
    "placeholders": { "produit": { "type": "String" } }
  },
  "montantXof": "{montant} XOF",
  "@montantXof": {
    "placeholders": { "montant": { "type": "num", "format": "decimalPattern" } }
  }
}
```

8b.3 Lint — no_hardcoded_strings

```
// analysis_options.yaml – règle custom
analyzer:
  plugins:
    - custom_lint

custom_lint:
  rules:
    - no_hardcoded_strings # Bloque tout String UI non via AppLocalizations

// ❌ INTERDIT – détecté par le lint
Text('Nouvelle commande')
AlertDialog(title: Text('Erreur'))

// ✅ CORRECT
Text(context.l10n.nouvelleCommande)
AlertDialog(title: Text(context.l10n.erreurGenerique))

// Exception : strings techniques (IDs, URLs, keys) sont exemptés
// Le lint vérifie uniquement les widgets Text, Tooltip, Semantics
```

8b.4 i18n dans le JSON config BDUI

Les labels dans la config JSON sont des clés i18n, pas des strings directs. Scalario Forge génère des clés, pas du texte brut.

```
// ❌ MAUVAIS – string hardcodé dans la config
{ "type": "KPICard", "props": { "label": "Ventes du jour" } }

// ✅ CORRECT – clé i18n dans la config
{ "type": "KPICard", "props": { "label_key": "kpi.ventes_jour" } }

// Flutter résout la clé au runtime selon la locale de l'user
final label = AppLocalizations.of(ctx)!.translate(config.labelKey);

// Scalario Forge génère des clés standard depuis le catalogue
// Si la clé n'existe pas – Scalario Forge la crée dans l'ARB
// Traduction humaine ou automatique (DeepL/Claude) ensuite
```

Règle fondamentale i18n

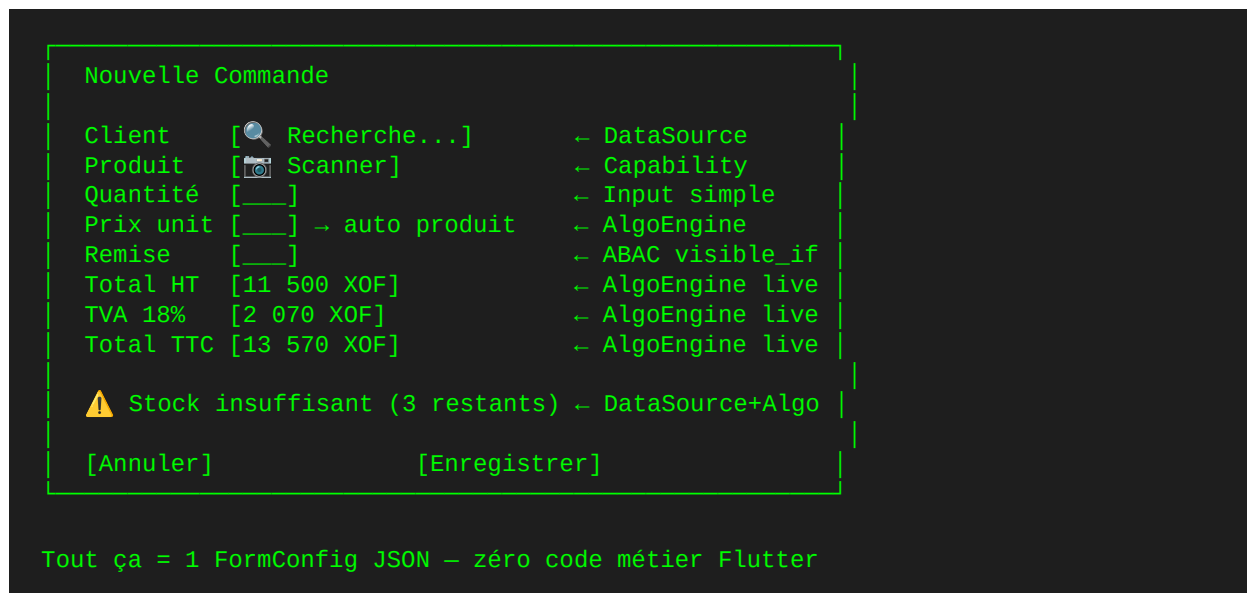
Aucun string visible par l'utilisateur n'est écrit en dur dans le code Flutter.
Aucun string visible n'est écrit en dur dans la config JSON du tenant.
Tout passe par les ARB files — le lint bloque toute violation dès la CI.

Scalaro Forge génère des label_key cohérents avec le catalogue i18n.
Les langues africaines (Bambara, Wolof...) sont ajoutées en Phase 2 sans toucher au code.

9. Scalario Form — Saisie & Validation

Le Scalario Form est l'engine sans lequel l'ERP ne peut pas recevoir de données. Ce n'est pas un simple composant UI — c'est un orchestrateur qui coordonne les 4 autres engines en temps réel pendant la saisie.

7.1 Ce que le FormEngine orchestre en temps réel



The screenshot shows a Flutter form titled "Nouvelle Commande" with the following fields and their associated engines:

- Client: [🔍 Recherche...] ← DataSource
- Produit: [📷 Scanner] ← Capability
- Quantité: [] ← Input simple
- Prix unit: [] → auto produit ← AlgoEngine
- Remise: [] ← ABAC visible_if
- Total HT: [11 500 XOF] ← AlgoEngine live
- TVA 18%: [2 070 XOF] ← AlgoEngine live
- Total TTC: [13 570 XOF] ← AlgoEngine live

Below the fields, there is a warning message: ⚠ Stock insuffisant (3 restants) ← DataSource+Algo.

At the bottom, there are two buttons: [Annuler] and [Enregistrer].

Tout ça = 1 FormConfig JSON – zéro code métier Flutter

7.2 Config JSON d'un formulaire

```
{ "form_id": "nouvelle_commande",
  "fields": [
    { "id": "produit", "type": "scan_or_search",
      "capability": "scanner",
      "datasource": { "source": "produits", "search_on": ["nom", "qr_code"] },
      "on_select": { "fill": { "prix_unitaire": "$produit.prix_vente" } },
      "output": "produit" },
    { "id": "quantite", "type": "number",
      "validation": { "min": 1, "max": "$stock_dispo",
        "message_max": "Stock insuffisant" } },
    { "id": "remise", "type": "number",
      "visible_if": { "role": ["MANAGER", "DG"] },
      "validation": { "max": 20 } },
    { "id": "total_ttc", "type": "computed",
      "formula": { "fn": "mul", "args": ["$total_ht", 1.18] },
      "format": "currency_xof" }
  ],
  "on_submit": { "pipeline": "creer_commande" }
}
```

7.3 Pourquoi c'est un engine à part

FormEngine — 4 raisons d'être distinct

1. Il orchestre les autres engines EN TEMPS RÉEL — chaque frappe peut déclencher AlgoEngine, chaque scan déclenche CapabilityRegistry, chaque recherche déclenche DataSourceRegistry.
2. Il a son propre state — valeurs courantes, état de validation, champs calculés, dépendances entre champs.
3. Il applique l'ABAC au niveau champ — visible selon rôle, readonly selon rôle, validation différente selon rôle.
4. Il est le seul point d'entrée des données utilisateur — toute donnée passe par sa validation avant d'atteindre le pipeline on_submit.

10. AlgoEngine — Fonctions Composables

L'AlgoEngine évalue des formules JSON composées de fonctions atomiques codées en vrai code. L'IA génère les compositions — elle ne code pas les fonctions de base.

8.1 Fonctions atomiques — codées une fois

```
// catalog/algo/primitives/ – fonctions typées, testées, immuables
export const primitives = {
  // Math
  add: (a: number, b: number) => a + b,
  sub: (a: number, b: number) => a - b,
  mul: (a: number, b: number) => a * b,
  div: (a: number, b: number) => { if(b===0) throw 'div/0'; return a/b },
  round: (n: number, d=2) => Math.round(n * 10**d) / 10**d,
  // Logique
  if: (cond: boolean, a: Value, b: Value) => cond ? a : b,
  gt: (a: number, b: number) => a > b,
  // Listes
  sum: (list: number[]) => list.reduce((a,b) => a+b, 0),
  avg: (list: number[]) => sum(list) / list.length,
  filter:(list: any[], field: string, val: any) => list.filter(i =>
i[field]===val),
  // Dates
  today: () => new Date(),
  diff_jours: (a: Date, b: Date) => Math.floor((b-a)/86400000),
  // Texte
  concat: (...args: string[]) => args.join(''),
}

// Chaque fonction déclare sa signature – Zod valide au déploiement
// Si add('5', 3) → erreur au déploiement, pas au runtime
```

8.2 Compositions — générées par l'IA

```
// L'IA génère ce JSON depuis la conversation
// 'Calculer le montant TTC avec remise éventuelle'
{
  "function_id": "calcul_facture",
  "formula": {
    "fn": "mul",
    "args": [
      { "fn": "sub", "args": [
        { "fn": "mul", "args": ["$quantite", "$prix_unitaire"] },
        { "fn": "mul", "args": ["$quantite", "$prix_unitaire",
          { "fn": "div", "args": ["$remise", 100] } ] }
      ] },
      1.18
    ]
  }
}
```

```

    }
  }

  // Mode debug — arbre d'évaluation complet
  AlgoEngine.eval(formula, inputs, { debug: true })
  // → { fn:'mul', steps:[{fn:'sub', result:10000}, 1.18], result:11800 }

```

8.3 Où les calculs s'exécutent

Flutter (offline)	NestJS (serveur)
Total d'une ligne commande	Calcul de paie complet
Montant TTC en temps réel	Commission vendeur mensuelle
Filtres visuels formulaire	Solde comptable certifié
Validation champ quantité	Clôture fiscale mensuelle
Stock affiché localement	Rapprochement bancaire

6 pièges à anticiper sur l'AlgoEngine

1. Formules profondes illisibles → debug step-by-step obligatoire (eval debug=true retourne l'arbre).
2. Typage strict — add('5', 3) doit échouer au déploiement via Zod, jamais au runtime en silencieux.
3. Ne jamais mélanger AlgoEngine et CapabilityRegistry — les algos sont purs, les capacités sont async/side-effect.
4. DataSourceRegistry nécessaire pour les agrégations — sum(ventes.montant) doit savoir où chercher 'ventes'.
5. Performance — memoization + compilation en closure Dart au premier appel pour 1000+ lignes calculées.
6. Tests — property-based testing sur chaque atomique (fast_check/glados), E2E sur les compositions.

11. Scalario Sense — CapabilityRegistry & Mobile Money

Scalario Sense est le registre des fonctionnalités hardware et système. Il inclut les APIs de paiement Mobile Money — Wave, Orange Money, MTN MoMo — qui sont des capacités critiques pour le marché africain.

9.1 Catalogue des capacités

```

catalog/capabilities/
├── input/
│   ├── barcode_scan.json    → output: { raw, type, format }
│   ├── photo_capture.json   → output: { image_base64, path }
│   ├── signature_capture.json → output: { image_base64, timestamp, signed }
│   ├── nfc_read.json        → output: { tag_id, data }
│   ├── voice_input.json     → output: { text, confidence }
│   └── document_scan.json   → output: { text_ocr, image_base64 }
├── output/
│   ├── printer_bluetooth.json → inputs: { template, data, copies }
│   ├── sms_send.json         → inputs: { to, message }
│   └── share_file.json       → inputs: { file_path, type }
├── location/
│   ├── gps_position.json    → output: { lat, lng, accuracy }
│   └── gps_track.json       → output: stream { lat, lng, timestamp }
├── auth/
│   └── biometrie.json       → output: { verified: boolean }
├── integration/
│   └── ← Connexions externes
│       ├── webhook_send.json → inputs: { url, payload, secret }
│       └── http_call.json    → inputs: { url, method, auth, params }
└── payment/
    └── ← Mobile Money – Phase 2
        ├── wave_pay.json    → output: { tx_id, status, montant }
        ├── orange_money.json → output: { tx_id, status, montant }
        └── mtn_momo.json     → output: { tx_id, status, montant }
  
```

9.2 Mobile Money — architecture détaillée (Phase 2)

Les APIs Mobile Money africaines (Wave, Orange Money, MTN MoMo) suivent des patterns similaires mais ont des SDKs distincts. L'architecture les abstrait derrière un contrat unique.

```

// Contrat unique – peu importe le provider
interface MobileMoneyResult {
    tx_id:    string;
  
```

```

    status:    'success' | 'pending' | 'failed';
    montant:   number;
    provider:  'wave' | 'orange_money' | 'mtn_momo';
    phone:     string;
    timestamp: Date;
  }

  // Dans un pipeline Scalario Flow
  {
    "id": "paiement",
    "registry": "sense",
    "fn": "wave_pay",
    "inputs": {
      "montant": "$commande.total_ttc",
      "phone":   "$client.telephone",
      "ref":     "$commande.ref"
    },
    "output": "paiement",
    "on_error": {
      "payment_failed": "notify",
      "payment_pending": "wait_webhook"
    }
  }
}

```

```

// NestJS – Webhook handler pour confirmation asynchrone
// Wave, Orange Money, MTN appellent ce webhook après paiement
@Post('webhooks/mobile-money')
@UseGuards(WebhookSignatureGuard) // vérifie la signature du provider
async handleMobileMoneyWebhook(@Body() payload: MobileMoneyWebhook) {
  const { tx_id, status, provider } = payload;

  // Met à jour le statut du paiement
  await this.paiementService.updateStatus(tx_id, status);

  // Déclenche Scalario Live → notifie l'app
  await this.scalarLive.emit('payment_confirmed', {
    tx_id, status, provider,
    tenant_id: await this.resolveTenant(payload)
  });

  // Si succès → déclenche le pipeline suivant (facturation auto)
  if (status === 'success') {
    await this.scalarFlow.trigger('post_payment', { tx_id });
  }
}

```

9.2 Structure Flutter — 3 couches

```

// Couche 1 – Contrat abstrait

```

```

abstract class Capability<I, O> {
    String get id;
    bool get isAvailable;
    Future<O> execute(I input);
    O get mock; // utilisé en dev/test — pas de vrai hardware
}

// Couche 2 — Implémentation concrète
class ScannerCapability extends Capability<void, ScanResult> {
    String get id => 'scanner';
    bool get isAvailable => Platform.isAndroid || Platform.isIOS;
    Future<ScanResult> execute(void _) async {
        final status = await Permission.camera.request();
        if (!status.isGranted) throw CapabilityError('permission_denied');
        final result = await MobileScannerController().scan();
        if (result == null) throw CapabilityError('scan_cancelled');
        return ScanResult(raw: result.rawValue ?? '', type: result.format.name);
    }
    ScanResult get mock => ScanResult(raw: 'PROD-00123', type: 'QR');
}

// Couche 3 — Registre avec mode dev
class CapabilityRegistry {
    static bool devMode = kDebugMode; // mock en dev, vrai en prod
    static Future<dynamic> invoke(String id, Map params) async {
        final cap = _capabilities[id];
        if (cap == null) throw CapabilityNotFound(id);
        if (devMode) return cap.mock; // dev → mock immédiat
        if (!cap.isAvailable) throw CapabilityUnavailable(id);
        return cap.execute(params);
    }
    // Retourne les caps disponibles → NestJS l'envoie à l'IA
    static Map<String, bool> availability() =>
        _capabilities.map((id, cap) => MapEntry(id, cap.isAvailable));
}

```

9.3 Webhooks sortants — Scalario → Externe

Les webhooks sortants sont une capability Scalario Sense. Ils permettent à un tenant de notifier des outils externes lors d'événements métier. Configurés dans le tenant_config par Scalario Forge — pas de code.

```

// catalog/capabilities/output/webhook_send.json
{
    "id": "webhook_send",
    "label": "Webhook sortant",
    "platform": ["all"],
    "async": true,
    "inputs": {
        "url": { "type": "string", "desc": "URL de destination" },
        "payload": { "type": "object", "desc": "Données à envoyer" },
    }
}

```



```

    "secret": { "type": "string", "desc": "HMAC secret pour signature" },
    "method": { "type": "string", "default": "POST" }
  },
  "output": { "status": "number", "response": "object" }
}

// Dans un pipeline Scalario Flow :
{
  "id": "notifier_comptabilite",
  "registry": "sense",
  "fn": "webhook_send",
  "inputs": {
    "url": "$tenant.webhooks.comptabilite_url",
    "payload": { "ref": "$commande.ref", "montant": "$commande.total" },
    "secret": "$tenant.webhooks.comptabilite_secret"
  },
  "on_error": { "http_error": "retry", "timeout": "notify_admin" }
}

// http_call – appel API externe générique
{
  "id": "sync_extern",
  "registry": "sense",
  "fn": "http_call",
  "inputs": {
    "url": "https://monapp.com/api/orders",
    "method": "GET",
    "auth": { "type": "api_key", "header": "X-API-Key",
              "value": "$tenant.integrations.mon_api_key" }
  },
  "output": "external_data"
}

```

Règle — pas de Scalario Connect séparé

Les intégrations externes ne créent pas un engine séparé.

Webhook sortant = capability `webhook_send` dans Scalario Sense.

Appel API externe = capability `http_call` dans Scalario Sense.

L'URL, le secret et le mapping sont dans `tenant_config.webhooks` — configurés par Scalario Forge.

Scalario Shield vérifie la signature HMAC sur chaque appel sortant.

9.4 Gestion des erreurs dans un pipeline

```

// Chaque step capability peut déclarer son comportement en cas d'erreur
{
  "id": "impression",
  "registry": "capability",
  "fn": "imprimante",

```

```
"on_error": {  
  "hardware_unavailable": "skip",    // continuer sans imprimer  
  "print_failed":        "retry",    // réessayer 1 fois  
  "bluetooth_off":       "notify"    // alerter l'utilisateur  
}  
}
```

12. DataSourceRegistry — Accès aux Données

Le DataSourceRegistry abstrait l'accès aux données des deux côtés — Drift offline dans Flutter et PostgreSQL online dans NestJS — avec le même contrat JSON.

10.1 Les 3 niveaux d'accès

Niveau	Usage	Exemple
1 — Config JSON simple	80% des cas — lire/filtrer une entité	{ "source": "commandes", "where": { "statut": "soumise" } }
2 — Agrégation JSON	KPIs et tableaux de bord	{ "aggregate": { "total": { "fn": "sum", "field": "montant" } } }
3 — SQL nommé catalogue	Rapports complexes, clôtures	{ "query_id": "rapport_mensuel_pharmacie" }

10.2 Catalogue SQL — queries nommées

```

catalog/queries/
├── commun/
│   ├── dashboard_kpis.sql
│   └── audit_trail.sql
├── pharmacie/
│   ├── rapport_ventes_medicaments.sql
│   ├── alerte_peremption.sql
│   └── ca_par_famille.sql
├── btp/
│   ├── avancement_chantier.sql
│   └── cout_materiaux.sql
└── finance/
    ├── balance_comptable.sql
    └── rapprochement_bancaire.sql

```

Règle : l'IA référence une query par son ID – elle ne génère jamais de SQL brut. Le SQL brut reste dans le catalogue – validé par toi, jamais exposé à l'IA.

10.3 Ce que PostgreSQL gère nativement (hors DataSourceRegistry)

PostgreSQL natif — la puissance brute

Stored Procedures → calcul de paie, clôture comptable, rapprochement bancaire (sensible, audité, transactionnel)

Triggers → updated_at automatique, audit log sur chaque UPDATE, sync queue pour offline (insert auto)

Vues matérialisées → dashboard agrégé rafraîchi toutes les heures, stats précalculées
RLS → filtre automatique par tenant + département — même si NestJS est bypassé

13. Scalario Live — Realtime Engine

Scalario Live est le 7ème engine. Il gère les événements venant DU SERVEUR vers l'app — contrairement aux 6 autres engines qui sont déclenchés par l'utilisateur. Sans Scalario Live, l'app ne peut pas se mettre à jour sans action de l'utilisateur.

13.1 Ce que Scalario Live gère

Événement serveur	Ce que Scalario Live fait
Commande soumise → DG doit valider	Notification push + badge sur menu Validation
Stock sous le seuil minimum	AlertBanner apparaît sur le dashboard
Autre user modifie une commande en cours	DataTable se rafraîchit automatiquement
Config mise à jour (v1.2 → v1.3)	Diff téléchargé + UI rechargée silencieusement
Message IA reçu (streaming)	Tokens affichés progressivement dans le chat
Session expirée côté serveur	Redirect login + message explicatif

13.2 Architecture — NestJS → Flutter

```

NestJS (événement métier déclenché) :
  Commande soumise → WorkflowEngine → besoin validation DG
  → WebSocket Gateway émet vers tous les DG connectés
  → Payload : { event: 'validation_required',
                data: { commande_id, montant, client } }

Scalario Live Flutter (écoute + dispatche) :
class ScalarioLive {
  final _channel = WebSocketChannel.connect(Uri.parse(wsUrl));

  void init() {
    _channel.stream.listen((raw) {
      final event = LiveEvent.fromJson(raw);
      switch (event.type) {
        case 'validation_required':
          // Badge sur le menu + notification locale
          BadgeManager.increment('validations');
          NotificationService.show(event.data);
          break;
        case 'data_updated':
          // Invalidier le cache Vault → refetch auto
          ScalarioVault.invalidate(event.data.source);
          break;
      }
    });
  }
}

```

```

        case 'config_updated':
            // Télécharger le diff de config
            ConfigSync.fetchDiff(event.data.newVersion);
            break;
        case 'alert_triggered':
            // Afficher une AlertBanner dans l'UI courante
            UIOverlay.show(AlertBanner.fromEvent(event));
            break;
    }
    });
}
}

```

13.3 Events configurés dans Scalario Pipe

```

// Les événements live sont déclarés dans la config tenant
// L'IA configure quels events déclenchent quelles réactions
{
  "live_events": [
    {
      "event": "validation_required",
      "condition": { "role": ["MANAGER", "DG"] },
      "reactions": [
        { "type": "badge", "target": "menu.validations" },
        { "type": "notification", "message": "Commande $montant XOF à valider" },
        { "type": "refresh", "source": "commandes" }
      ]
    },
    {
      "event": "stock_critical",
      "condition": { "role": ["PHARMACIEN", "MANAGER", "DG"] },
      "reactions": [
        { "type": "alert", "component": "AlertBanner",
          "variant": "danger", "message": "$produit en rupture" }
      ]
    }
  ]
}

```

13.4 Webhooks entrants — Externe → Scalario

Les webhooks entrants sont des events Scalario Live. Un outil externe appelle un endpoint NestJS → Scalario Live vérifie la signature → déclenche un pipeline Scalario Flow. Configurés par Scalario Forge dans le tenant_config.

```

// Config tenant – webhook entrant configuré par Scalario Forge
{

```

```

    "live_events": [
      {
        "event": "webhook_inbound",
        "source": "woocommerce",
        "secret": "$tenant.webhooks.woocommerce_secret",
        "mapping": {
          "ref": "$payload.order_number",
          "montant": "$payload.total",
          "client": "$payload.billing.email",
          "lignes": "$payload.line_items"
        },
        "on_receive": "pipeline_creer_commande",
        "on_error": "notify_admin"
      }
    ]
  }
}

// NestJS — endpoint générique webhooks entrants
@Post('live/webhooks/:tenantHandle/:sourceId')
@UseGuards(WebhookSignatureGuard) // vérifie HMAC secret
async handleInbound(
  @Param('tenantHandle') handle: string,
  @Param('sourceId') sourceId: string,
  @Body() payload: any
) {
  const tenant = await this.tenantService.findByHandle(handle);
  const connector = tenant.config.live_events
    .find(e => e.source === sourceId);

  // Transforme le payload selon le mapping config
  const normalized = await this.calc
    .transform(payload, connector.mapping);

  // Déclenche le pipeline Scalario Flow configuré
  await this.scalariorFlow.trigger(
    connector.on_receive,
    normalized,
    { tenantId: tenant.id }
  );
}

// URL webhook à donner à WooCommerce :
// https://api.scalarior.app/live/webhooks/@pharmacie-kossyam/woocommerce

```

Règle — pas de Scalario Connect séparé

Webhook entrant = event webhook_inbound dans Scalario Live.

L'endpoint NestJS est générique — le tenant_handle et le sourceId identifient le tenant et la source.

La signature HMAC est vérifiée par Scalario Shield avant tout traitement.

Le mapping payload → données Scalario est dans tenant_config — configuré par Scalario Forge.

Le pipeline déclenché est un Scalario Pipe standard — pas de code spécifique par connecteur.

13.5 Push Notifications hors app — FCM & APN

Scalario Live gère les notifications in-app via WebSocket. Quand l'app est FERMÉE, FCM (Android) et APN (iOS) prennent le relais pour livrer les notifications push.

Cas	Mécanisme
App ouverte — user actif	Scalario Live WebSocket — temps réel immédiat
App en arrière-plan	FCM/APN + WebSocket au retour au premier plan
App fermée	FCM (Android) ou APN (iOS) — notification système
Flutter Web	Web Push API (VAPID) — navigateur fermé possible

```
// NestJS — envoi via FCM quand app fermée
@Injectable()
export class PushService {
  async notify(userId: string, payload: PushPayload) {
    const user = await this.userRepo.findOne(userId);

    // WebSocket si connecté (app ouverte)
    if (this.scalariorLive.isConnected(userId)) {
      return this.scalariorLive.emit(userId, payload);
    }

    // FCM/APN si app fermée
    if (user.fcmToken) {
      await this.fcm.send({
        token: user.fcmToken,
        notification: { title: payload.title, body: payload.body },
        data: { event: payload.event, tenant_id: user.tenantId }
      });
    }
  }
}

// Flutter — enregistrement du token FCM/APN au login
final fcmToken = await FirebaseMessaging.instance.getToken();
await apiClient.post('/users/push-token', { token: fcmToken });

// Flutter — réception notification app fermée
FirebaseMessaging.onBackgroundMessage(_handleBackgroundMessage);
// → ouvre l'écran concerné selon event.data
```


13.6 Navigation — dans Scalario Canvas, pas un engine

La navigation n'est pas un engine car elle ne traite pas de données métier. C'est une déclaration JSON évaluée par go_router Flutter.

```
// NavigationConfig dans le JSON tenant_config
{
  "navigation": {
    "type": "bottom_nav",      // bottom_nav | drawer | tabs
    "items": [
      { "id": "dashboard", "icon": "home", "label": "Accueil",
        "visible_if": { "role": ["*"] } },
      { "id": "commandes", "icon": "cart", "label": "Commandes",
        "badge_source": "commandes.pending_count",
        "visible_if": { "role": ["VENDEUR", "MANAGER", "DG"] } },
      { "id": "validations", "icon": "check", "label": "Validation",
        "badge_source": "live.validation_count",
        "visible_if": { "role": ["MANAGER", "DG"] } },
      { "id": "rapports", "icon": "chart", "label": "Rapports",
        "visible_if": { "role": ["COMPTABLE", "DG"] } }
    ]
  }
}

// go_router Flutter lit ce JSON — zéro code de navigation écrit
// Les badges sont mis à jour par Scalario Live en temps réel
```

13.7 State & Error — couche technique Flutter

Pas dans la nomenclature — mais documenté ici

Riverpod gère l'état global : loading states, error boundaries, retry strategies, session expirée. C'est de l'infrastructure Flutter standard — invisible pour l'IA et le JSON config. Chaque engine expose des états observables (ScalarioVault.loading, ScalarioSense.error...). Scalario Canvas réagit à ces états pour afficher LoadingSpinner, ErrorBanner, RetryButton. Non nommé publiquement car c'est du plomberie technique, pas de la logique métier.

14. Scalario Flow — ActionEngine

Scalario Flow est l'orchestrateur central. Il exécute des pipelines JSON en coordonnant les 6 engines via des variables partagées. Chaque step déclare son engine (registry) et sa fonction.

11.1 Exemple complet — 4 engines en 1 pipeline

```
{ "action": "reception_livraison",
  "trigger": "bouton_reception",
  "steps": [
    { "id": "scan",      "registry": "capability",  "fn": "scanner",
      "output": "qr" },
    { "id": "commande", "registry": "datasource",
      "source": "commandes", "where": { "qr_code": "$qr.raw" },
      "output": "cmd" },
    { "id": "photo",    "registry": "capability",  "fn": "camera",
      "output": "photo" },
    { "id": "signature", "registry": "capability",  "fn": "signature",
      "inputs": { "label": "Signature : $cmd.client.nom" },
      "output": "sig" },
    { "id": "retard",   "registry": "algo",
      "fn": "diff_jours", "args": ["$cmd.date_prevue", "today()"],
      "output": "jours" },
    { "id": "save",     "registry": "datasource", "action": "update",
      "source": "livraisons",
      "data": { "commande_id": "$cmd.id", "photo": "$photo.image_base64",
                "signature": "$sig.image_base64", "statut": "livree",
                "retard_jours": "$jours" } },
    { "id": "alerte",  "registry": "component",  "fn": "AlertBanner",
      "props": { "message": "Livraison confirmée – retard $jours j",
                  "status": { "fn": "if", "args": [
                      { "fn": "gt", "args": ["$jours", 0] }, "warning", "success" ] } } },
    { "id": "print",   "registry": "capability",  "fn": "imprimante",
      "inputs": { "template": "bon_livraison", "data": "$cmd" },
      "on_error": { "hardware_unavailable": "skip" } }
  ]
}
```

14.1 Code Flutter — Scalario Flow

```
// Scalario Flow – dispatch vers les 6 engines
class ScalarioFlow {
  final variables = <String, dynamic>{};

  Future<void> run(List<Step> steps, BuildContext ctx) async {
    for (final step in steps) {
      final resolved = resolveParams(step.params, variables);
      try {
        final result = await switch (step.registry) {
```

```
        'canvas'      => ScalarioCanvas.render(step.fn, resolved, ctx),
        'form'       => ScalarioForm.show(step.formId, resolved, ctx),
        'calc'       => ScalarioCalc.evaluate(step.fn, resolved),
        'sense'      => ScalarioSense.invoke(step.fn, resolved),
        'vault'      => ScalarioVault.execute(step, resolved),
        'live'       => ScalarioLive.emit(step.event, resolved),
        _ => throw UnknownRegistry(step.registry),
    };
    if (step.output != null) variables[step.output!] = result.data;
} catch(e) {
    await handleError(step, e); // on_error strategies
}
}
}
```

PARTIE 4 — BDUI & Design System

14. Backend-Driven UI — Principe & APK Universelle

12.1 L'APK universelle — le moteur BDUI vide

L'app Flutter est publiée une seule fois sur les stores. Elle ne contient aucune logique métier, aucune config client. C'est un moteur de rendu vide qui s'anime au login.

```
L'APK contient (identique pour tous les clients) :
  BDUI Engine — ComponentRegistry, RuleEvaluator, LayoutResolver
  Scalario Flow + 7 engines
  Material 3 + Design Tokens
  Écran de login universel
  Drift/Isar (cache local vide)
  CapabilityRegistry (scanner, GPS, print...)
```

```
L'APK NE contient PAS :
  Aucune config client
  Aucune logique métier
  Aucune donnée
  Aucune référence à un tenant
```

```
Au login — NestJS pousse tout :
  token + UserContext (rôle, dept, attributs)
  TenantConfig (screens, modules, rules, pipelines)
  configVersion (pour sync future)
```

```
Flutter stocke en Drift + rend immédiatement :
  BDUIEngine.render(config: auth.config, user: auth.user)
  → L'UI de son ERP apparaît selon son rôle
```

12.2 Le flux login → UI

1. User ouvre l'app (APK vide)
 - Écran login universel
2. User entre email + mot de passe
 - POST /auth/login
3. NestJS identifie :
 - Quel tenant (pharmacie_kossyam ?)
 - Quel rôle (VENDEUR ? MANAGER ?)
 - Quels attributs ABAC (dept, plafond...)
 - Quelle version de config (v1.3 ?)
4. NestJS retourne :
 - JWT token
 - Config complète du tenant filtrée par rôle
 - Uniquement les screens que ce rôle peut voir

```

5. Flutter :
  → Stocke config dans Drift (offline ready)
  → BDUIEngine.render() → UI apparaît
  → Vendeur voit : Commandes, Stock, Clients
  → Comptable voit : Factures, Rapports, Paiements
  → DG voit : Dashboard consolidé + tout

6. Même APK — 3 utilisateurs — 3 UI différentes
   Même APK — 100 clients — 100 ERP différents

```

12.3 Offline — résilience, pas infrastructure client

L'offline dans le modèle SaaS

L'offline n'est pas 'le client héberge son serveur' — c'est la résilience de l'app.

Après le premier login, la config est cachée dans Drift sur l'appareil.

Sans connexion : l'app fonctionne normalement — BDUI, FormEngine, Capabilities.

Quand la connexion revient : sync CRDT avec TES serveurs — données réconciliées.

Le client ne gère rien — il utilise l'app. Toi tu gères l'infrastructure.

```

Sync config (légère) :
  App au démarrage : ma version locale = v1.2
  Serveur : nouvelle version v1.3 disponible
  App télécharge uniquement le diff
  UI mise à jour — zéro réinstallation

Sync données (CRDT) :
  Livreur crée bon de livraison offline
  Connexion revient → merge CRDT automatique
  Audit log sur TES serveurs

```

12.4 Le principe BDUI

ERP classique	Scalario BDUI
APK différente par client	1 Scalario universel pour tous
50 écrans Flutter codés	1 moteur + configs JSON illimitées
Modifier = retoucher le code	Modifier = changer le JSON en admin
Update App Store pour changer l'UI	Diff de config — zéro réinstallation
1 rendu fixe par composant	1 composant + N variantes contextuelles

12.5 ComponentConfig — le contrat JSON complet

Le JSON Schema BDUI intègre maintenant le champ variant. C'est le contrat entre NestJS, Scalario Forge et Scalario Canvas.

```
// TypeScript – contrat complet d'un composant BDUI
interface ComponentConfig {
  type:      string;           // 'KPICard' | 'DataTable' | ...
  variant:   string;           // 'default' | 'compact' | 'auto' | ...
  props:     Record<string, any>;
  visible_if?: Rule;           // ABAC – qui peut voir ce composant
  actions?:  ActionStep[];     // ce que le composant peut déclencher
  children?: ComponentConfig[]; // composition de composants
}

// Exemple complet :
{
  "type": "KPICard",
  "variant": "auto",           // Flutter adapte au contexte
  "props": {
    "label": "Ruptures stock",
    "value_source": "stock.ruptures",
    "status_source": "stock.status",
    "format": "number"
  },
  "visible_if": { "role": ["MANAGER", "PHARMACIEN", "DG"] },
  "actions": [
    { "on": "tap", "pipeline": "voir_ruptures_detail" }
  ]
}
```

12.6 Ce que tu maquettes avec Widgetbook

Widgetbook – ton Figma vivant : chaque composant × chaque variante × chaque état

```
├─ Tokens (couleurs, spacing, typo)
├─ Composants/
│   ├─ KPICard/
│   │   ├─ default (normal, warning, danger, loading)
│   │   ├─ compact (mobile 4 KPIs)
│   │   ├─ with-icon (icône + statut)
│   │   ├─ hero (seul sur desktop)
│   │   └─ auto (simuler mobile DG, mobile vendeur, desktop...)
│   └─ DataTable/
│       ├─ default (colonnes standards)
│       ├─ compact (haute densité)
│       └─ card-list (mobile)
└─ AlertBanner/
    └─ info · success · warning · danger · dismissible
```

```

├── Layouts/ (4-5 max, codés une fois)
│   ├── DashboardLayout
│   ├── ListLayout
│   ├── FormLayout
│   └── DetailLayout
├── Sandbox JSON/ (valide l'engine + variante auto)
│   ├── pharmacie_dashboard_mobile.json
│   ├── pharmacie_dashboard_desktop.json
│   └── btp_chantiers_mobile.json

```

12.6 Ordre de développement

Semaine	Tâche	Livrable
S1	JSON Schema BDUI (ComponentConfig, Rule, ActionStep)	Contrat TypeScript/Dart validé
S2	Material 3 + ThemeData custom + Design Tokens	Design System natif Flutter prêt
S3	Widgetbook + 5 composants de base	KPICard, DataTable, AlertBanner, ListTile, FAB
S4	BDUI Engine + Login flow (APK universelle)	Sandbox JSON → rendu selon rôle
S5-6	FormEngine + AlgoEngine (atomiques)	Formulaires + calculs en temps réel
S7-8	CapabilityRegistry + ActionEngine	Pipelines scan → compute → save fonctionnels
M2	Flutter Web (admin + Business Profile + Demo Space)	Config chat + preview identique app
M2	Adaptive layouts desktop	UI adaptée grands écrans — managers, DG

15. UX par Métier & Multi-Business

13.1 UX Profiles par secteur

L'IA ne crée pas de nouveaux patterns UX. Elle choisit parmi des profiles validés par secteur. Ces profiles garantissent que l'UI est cohérente avec les habitudes métier du client.

```

catalog/
├── ux_profiles/
│   ├── pharmacie/
│   │   ├── layout_rules.json    ← règles spécifiques pharma
│   │   ├── screen_templates.json ← écrans types (caisse, stock)
│   │   └── ux_patterns.json     ← workflows validés
│   ├── btp/
│   ├── cabinet_medical/
│   ├── commerce_general/
│   └── _base/                  ← règles communes à tous

```

13.2 Multi-Business — 3 approches

Approche	Description	Quand l'utiliser
Espaces séparés	Chaque business charge son UX Profile complet. Switch conscient.	Métiers très différents
Fusion par rôle	DG voit une vue consolidée. Comptable voit les finances des 2.	Même direction, reporting unifié
Modules partagés	L'IA détecte les modules communs et propose fusion.	Modules réellement identiques

4 règles de fusion non négociables

Règle 1 — Séparation par défaut : 2 métiers = 2 espaces distincts sauf demande explicite.

Règle 2 — Fusion sur modules identiques seulement : Stock pharmacie ≠ Stock BTP (unités différentes).

Règle 3 — Le rôle prime sur le métier : DG multi-business → toujours UX 'direction'.

Règle 4 — Jamais de pattern hybride inventé : si pas de pattern de fusion → espaces séparés.

PARTIE 5 — Modules ERP

16. Approche Modules — 6 Moteurs + JSON

14.1 Le vrai problème

ERP classique	Ton ERP BDUI
50 modules codés en dur	6 moteurs génériques + configs JSON
Personnalisation = modifier le code	Personnalisation = surcharger le JSON
Nouveau métier = nouveau dev	Nouveau métier = nouveau fichier JSON
50 écrans Flutter	1 moteur Flutter qui rend tout

14.2 Les 6 moteurs génériques — codés une fois

Moteur	Rôle	Exemple d'usage
ModuleList	N'importe quelle liste d'entités	Commandes, Stock, Clients, Factures
ModuleForm	N'importe quel formulaire	Nouvelle commande, Fiche employé
ModuleDetail	N'importe quelle fiche détail	Détail commande, Profil client
ModuleReport	N'importe quel rapport/export	CA mensuel, Rapport paie
ModuleKanban	N'importe quel workflow en colonnes	Congés, Chantiers, Support
ModuleDashboard	KPIs de n'importe quelle source	Dashboard DG, Stats vendeur

14.3 Catalogue de modules standards

```

catalog/modules/
├── gestion/
│   ├── commandes.json    (ModuleList + workflow approval)
│   ├── stock.json        (ModuleList + alertes seuils)
│   ├── clients.json      (ModuleList + fiche détail)
│   └── fournisseurs.json (ModuleList + fiche détail)
├── finance/
│   ├── factures.json     (ModuleList + génération PDF)
│   ├── paiements.json    (ModuleList + réconciliation)
│   └── rapports_fin.json  (ModuleDashboard + export)
├── rh/
│   ├── employes.json     (ModuleList + fiche)
│   ├── conges.json       (ModuleKanban + workflow)
│   └── paie.json          (ModuleReport + calculs)
└── operations/

```

```

├─ livraisons.json      (ModuleList + capabilities: scan, sign, print)
├─ planning.json       (ModuleKanban)
└─ chantiers.json      (ModuleDashboard)

```

14.4 Héritage — chaque client surcharge uniquement ce qui diffère

```

// Module standard du catalogue
{ "module_id": "commandes", "engine": "ModuleList",
  "actions": { "approve": { "condition": "montant > 0" } } }

// Override tenant Pharmacie Kosyiam
{ "module_id": "commandes",
  "inherits": "catalog/modules/gestion/commandes",
  "overrides": {
    "label": "Bons de commande",
    "schema": {
      "ordonnance": { "type": "file", "label": "Ordonnance", "required": true }
    },
    "actions": {
      "approve": { "condition": "montant > 100000", "roles": ["PHARMACIEN"] }
    }
  }
}

// L'IA génère cet override depuis la conversation :
// 'chaque commande doit avoir une ordonnance'
// 'validée par le pharmacien si > 100k XOF'

```

Élément	Comment	Résultat
6 moteurs Flutter	Codés une fois	Rendent tout
~20 modules standards catalogue	Fichiers JSON	Base partagée tous clients
Modules spécifiques client	JSON override généré par l'IA	Illimité, zéro code
Nouveaux modules	Nouveau fichier JSON catalogue	Zéro code Flutter/NestJS
Nouveau type de métier	Nouveaux fichiers JSON ux_profiles + modules	Zéro code

PARTIE 6 — Configuration IA — Du Business au Go Live

17. Business Profile Document

Au lieu d'une conversation interactive longue qui fait procrastiner, le client remplit un document structuré à son rythme. L'IA l'analyse ensuite et pose seulement les questions qui changent vraiment quelque chose.

15.1 Structure du Business Profile

Sections du formulaire (remplissage libre, sauvegarde auto) :

1. VOTRE ACTIVITÉ
Nom, secteur (liste), ville, taille (1-10 / 11-50 / 50+)
Décrivez votre activité principale : [texte libre]
2. VOS ÉQUIPES
Listez vos types d'employés et ce qu'ils font : [texte libre]
Exemple : 'Vendeurs (3) : prennent les commandes et gèrent le stock'
3. VOS PROCESSUS PRINCIPAUX
Comment se passe une vente typique ? [texte libre]
Y a-t-il des validations ou approbations requises ? [texte libre]
4. VOS OUTILS ACTUELS
☒ Excel ☒ Cahier ☐ Logiciel ☐ Autre
5. SPÉCIFICITÉS IMPORTANTES
Travaillez-vous souvent sans internet ? [oui/non]
Vos équipes ont-elles des appareils mobiles ? [oui + type]
Y a-t-il des règles métier importantes ? [texte libre]

15.2 Ce que l'IA extrait automatiquement

```
// Input : 'On vend des médicaments en gros à des pharmacies.
//          Le DG valide les commandes > 500k XOF.
//          Les livreurs ont des tablettes Android sans internet.'

// Extraction automatique (sans question) :
✓ Secteur          → pharmacie_gros
✓ Rôles            → vendeur(3), livreur(2), comptable(1), DG(1)
✓ Workflow         → commande→vérif→validation→livraison→facturation
✓ Règle ABAC       → validation DG si montant > 500k XOF
✓ Capabilities     → imprimante Bluetooth, signature tactile
✓ Offline critique → livreurs sans réseau → offline-first
✓ Devices          → tablettes Android
✓ Modules          → stock, commandes, livraisons, facturation

// Non extrait → questions de précision :
```

```
? Gérez-vous les dates de péremption des médicaments ?  
? Les livreurs peuvent-ils voir les prix des commandes ?  
? La comptable peut-elle modifier une facture déjà émise ?
```

15.3 Questions de précision — format guidé (max 7)

Règle anti-procrastination

Jamais plus de 7 questions — jamais de texte libre dans les questions de précision.

Toujours des choix multiples — le client clique, il ne rédige pas.

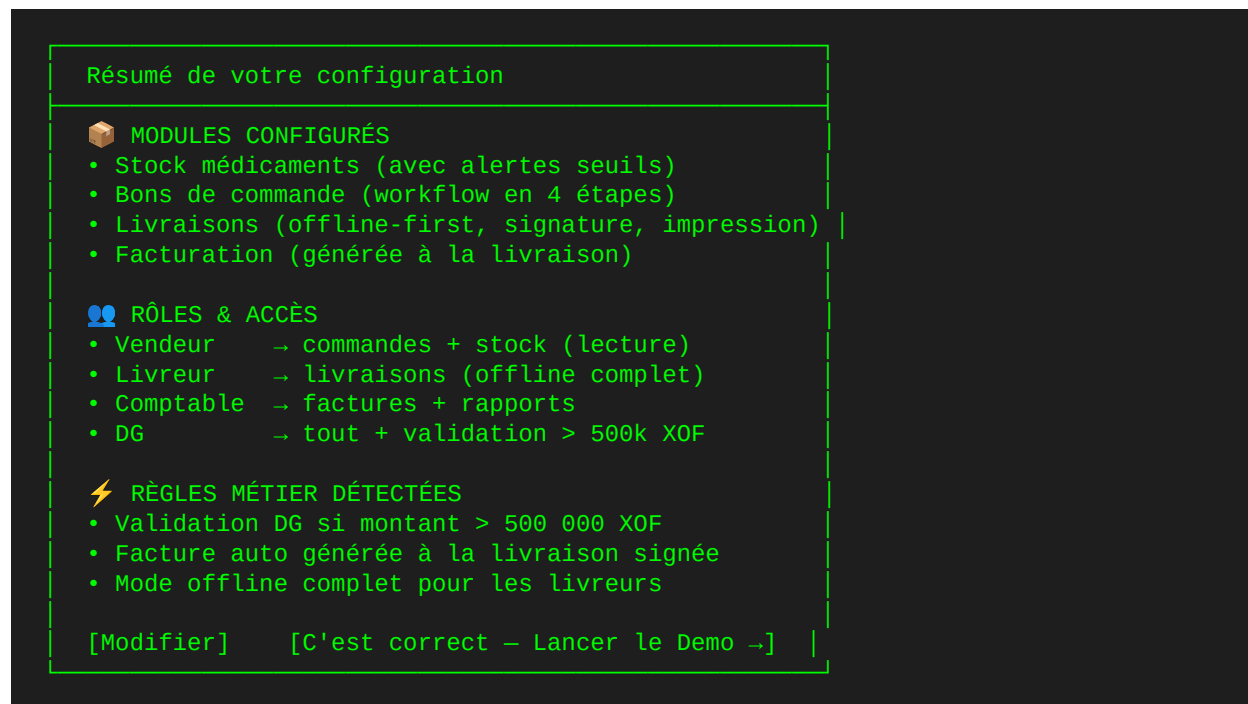
Chaque question doit changer quelque chose d'important dans la config.

Si une info peut être corrigée facilement après → ne pas la demander maintenant.

Objectif : répondre en moins de 3 minutes.

18. Analyse IA + Résumé Technique

Après le Business Profile et les questions de précision, l'IA génère un résumé en langage naturel que le client valide — pas le JSON technique.



16.1 Anti-hallucination — 6 couches de protection

Couche	Mécanisme	Ce qu'elle détecte
1 — Structure	Instructor + Zod	Fonctions inexistantes, JSON invalide — rejeté au déploiement
2 — Logique	ConfigValidator NestJS	Types incohérents, refs manquantes, cycles DAG
3 — Exécution	Dry Run (données fictives)	Bugs runtime avant production
4 — Métier	Résumé langage naturel	Mauvaise compréhension — le client valide en français
5 — Test réel	Demo Space sandbox	Le client teste lui-même avec données réalistes
6 — Filet	Rollback 30 jours	Tout bug passé en production — retour en 1 clic

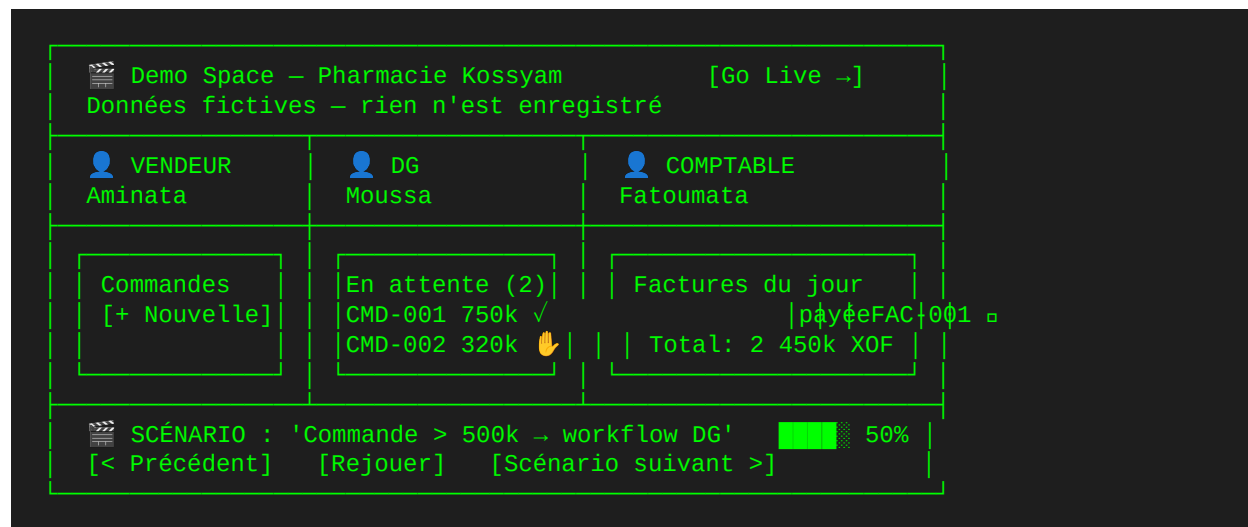
19. Demo Space — Test Multi-Rôles Simultané

Avant le go live, le client entre dans un espace sandbox avec des données fictives réalistes et peut voir tous les rôles simultanément — le mode Réalisateur.

17.1 Les 3 modes du Demo Space

Mode	Description
 Joueur	Le client joue UN rôle. Les autres sont simulés par l'IA automatiquement.
 Réalisateur	Tous les rôles visibles simultanément. Interactions possibles avec chacun.
 Formation	Guidé étape par étape. Idéal pour former les futurs utilisateurs.

17.2 L'écran multi-rôles simultané



17.3 Ajustements en temps réel depuis le Demo Space

Client dans le Demo Space :

"Le seuil de validation devrait être 200k, pas 500k"

- L'IA modifie la config en temps réel
- Le scénario se rejoue avec la nouvelle règle
- Le client voit la différence immédiatement
- L'ajustement est sauvegardé dans la config finale

"Le comptable ne devrait pas voir les commandes en cours"
→ ABAC ajusté → écran comptable mis à jour instantanément

Chaque correction est mémorisée.
La config finale intègre tous les ajustements du Demo Space.

17.4 Flux complet — du Business Profile au Go Live

Jour 1 (30 min, à son rythme)
→ Client remplit le Business Profile
→ Sauvegarde auto, peut revenir le lendemain

Jour 1 (2 min après envoi)
→ L'IA analyse + génère résumé technique
→ 3 à 7 questions de précision (choix multiples)
→ Config générée + validée (Instructor + Dry Run)

Jour 1 (1-2 heures — Demo Space)
→ Données fictives réalistes générées
→ Mode Réalisateur — tous rôles visibles
→ Scénarios guidés par l'IA
→ Ajustements via chat intégré

Jour 2-7 (optionnel — affinage)
→ Teams internes testent le Demo Space
→ Mode Formation pour les futurs utilisateurs

Jour 7 — Go Live
→ Import données existantes (Excel/CSV)
→ Config validée → déployée
→ Rollback possible 30 jours

PARTIE 7 — Sécurité — RBAC + ABAC + RLS

20. RBAC + ABAC — Sécurité Multi-Couche

18.1 RBAC vs ABAC

RBAC (Role-Based)	ABAC (Attribute-Based)
Rôle → Permission simple	User + Ressource + Contexte → Décision
MANAGER voit les factures	MANAGER voit les factures DE SON DEPT si montant < 500k XOF
Simple, NestJS Guards natifs	Granulaire, CASL + Casbin + règles dynamiques

18.2 Les 5 couches de sécurité

```
Requête utilisateur
  ↓ Layer 1 : JWT Guard (NestJS) — authentication
  ↓ Layer 2 : RBAC Guard — le rôle a-t-il accès à cette route ?
  ↓ Layer 3 : ABAC CASL — les attributs autorisent-ils cet objet ?
  ↓ Layer 4 : pgvector filter — RAG filtrée par dept/rôle (AVANT le LLM)
  ↓ Layer 5 : PostgreSQL RLS — sécurité au niveau base de données
  → Données retournées (déjà filtrées — le LLM ne voit jamais de données brutes)
```

18.3 Règle critique — le LLM ne décide pas des permissions

NE JAMAIS laisser le LLM gérer les permissions

FAUX : Envoyer toutes les données au LLM et dire 'montre seulement ce que l'user peut voir'.
CORRECT : Filtrer les données AVANT d'appeler le LLM — le modèle reçoit uniquement les données autorisées.

Le LLM est un moteur de traitement, pas un garde de sécurité.

Chaque query LLM est auditée : qui a demandé quoi, quand, avec quelles données.

18.4 RLS PostgreSQL — dernière ligne de défense

```
-- Même si NestJS est bypassé, la DB protège
ALTER TABLE {tenant}.commandes ENABLE ROW LEVEL SECURITY;

CREATE POLICY commandes_dept_policy ON {tenant}.commandes
  USING (
    departement_id = current_setting('app.user_dept_id')::uuid
    OR current_setting('app.user_role') IN ('DG', 'ADMIN')
  );

-- NestJS injecte le contexte avant chaque query
```

```
await entityManager.query(  
  `SET LOCAL app.user_dept_id = '${user.deptId}';  
  SET LOCAL app.user_role    = '${user.role}';`  
);
```

PARTIE 8 — Backend & Base de Données

21. Architecture NestJS

19.1 Structure des modules

```

src/
├── core/                                ← Global, chargé une fois
│   ├── auth/                          JWT, OAuth2, sessions
│   ├── rbac/                          Guards, Decorators, Roles
│   ├── abac/                          CASL abilities, règles dynamiques
│   ├── tenant/                        Résolution du tenant par requête
│   └── audit/                         Log chaque action sensible
├── engines/                            ← Les 6 engines côté NestJS
│   ├── algo/                         AlgoEngine TypeScript
│   ├── datasource/                   DataSourceRegistry + queries
│   ├── action/                       ActionEngine pipeline executor
│   └── workflow/                     FSM + DAG validator
├── bdui/                              ← Génère le JSON UI
│   ├── bdui.service.ts               Charge config + applique ABAC
│   ├── bdui.resolver.ts              Résout les data sources
│   └── bdui.validator.ts              Valide le JSON avant sauvegarde
├── config-agent/                      ← Onboarding IA
│   ├── chat.gateway.ts               WebSocket + SSE
│   ├── extraction.service.ts          LLM → ERPConfig structurée
│   └── tenant-generator.ts            Génère la config JSON finale
├── modules/                           ← Modules ERP génériques
│   └── [commandes|stock|facturation|rh|rapports]/
└── ai/                               ← Pont vers FastAPI
    ├── ai.client.ts                  HTTP client vers FastAPI
    └── streaming.service.ts           SSE relay FastAPI → Flutter
  
```

19.2 Flux d'une requête — 6 couches

Flutter : GET /api/commandes

1. TenantMiddleware → lit X-Tenant-ID, charge config depuis Redis
2. JwtGuard → vérifie token, extrait user + roles + attributes
3. RbacGuard → @Roles('MANAGER','VENDEUR') → accès à la route ?
4. AbacInterceptor → CASL ability + filtre SQL selon attributs
5. CommandesService → query TypeORM avec filtre ABAC appliqué
6. AuditInterceptor → log : qui, quoi, quand, résultat

19.3 Documentation API — @nestjs/swagger

Swagger documente automatiquement l'API Scalario. En Phase 1, c'est l'outil interne de Scalario Labs. En Phase 3, c'est la documentation publique pour les intégrateurs.

```
// Installation
npm install @nestjs/swagger swagger-ui-express

// main.ts — setup Swagger
const config = new DocumentBuilder()
  .setTitle('Scalario API')
  .setVersion('1.0')
  .addBearerAuth()
  .addApiKey({ type: 'apiKey', name: 'X-Tenant-ID', in: 'header' })
  .build();
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api/docs', app, document);

// Annotation sur chaque controller
@ApiTags('Commandes')
@ApiBearerAuth()
@ApiHeader({ name: 'X-Tenant-ID', required: true,
  description: 'Identifiant du tenant Scalario' })
@Controller('commandes')
export class CommandesController {

  @ApiOperation({ summary: 'Lister les commandes du tenant' })
  @ApiQuery({ name: 'statut', enum: CommandeStatut, required: false })
  @ApiResponse({ status: 200, type: [CommandeDto] })
  @Get()
  findAll(@Query() query: CommandeQueryDto) { ... }
}

// Disponible sur : https://api.scalar.io/app/api/docs
// En dev local : http://localhost:3000/api/docs
```


22. Database Schema — PostgreSQL Multi-Tenant

20.1 Approche Schema-per-Tenant

Schema public (partagé)	Schema {tenant_id} (isolé)
public.tenants	{tenant}.commandes
public.tenant_configs (JSONB)	{tenant}.stock
public.users	{tenant}.factures
public.audit_logs	{tenant}.clients_erp
public.embeddings_index	{tenant}.embeddings (pgvector)

20.2 Table tenant_configs — cœur du BDUI

```
CREATE TABLE public.tenant_configs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES public.tenants(id),
  version     TEXT NOT NULL DEFAULT '1.0.0',
  metier      JSONB NOT NULL,      -- ['pharmacie', 'btp']
  modules     JSONB NOT NULL,      -- modules activés + overrides
  roles       JSONB NOT NULL,      -- définitions RBAC/ABAC
  ux_profiles JSONB NOT NULL,      -- règles UI par rôle
  screens     JSONB NOT NULL,      -- config BDUI des écrans
  workflows   JSONB NOT NULL,      -- FSM des workflows
  pipelines   JSONB NOT NULL,      -- ActionEngine pipelines
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Versioning pour sync offline
CREATE TABLE public.tenant_config_history (
  id UUID, tenant_id UUID, version TEXT,
  config JSONB, diff JSONB, changed_at TIMESTAMPTZ
);
```

20.3 pgvector + Hybrid Search

```
-- Table embeddings dans chaque schema tenant
CREATE TABLE {tenant}.embeddings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  content     TEXT NOT NULL,
  metadata    JSONB,               -- { module, dept_id, roles }
  embedding   vector(1536),        -- Claude / Cohere embeddings
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Index vectoriel pour recherche sémantique
CREATE INDEX ON {tenant}.embeddings
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

-- Index Full-Text pour BM25 (hybrid search)
CREATE INDEX ON {tenant}.embeddings
  USING gin(to_tsvector('french', content));
```

PARTIE 9 — Moteur IA — RAG, Mémoire & Config Agent

23. FastAPI — Microservice IA

```

ai-service/
├── routers/
│   ├── chat.py          SSE streaming token par token
│   ├── rag.py           Hybrid search + reranking
│   ├── config_agent.py  Extraction ERPConfig depuis Business Profile
│   └── embeddings.py    Génération + indexation Docling
├── agents/
│   ├── config_agent.py  LangChain + Instructor → ERPConfig
│   └── query_agent.py   Réponses questions métier
└── rag/
    ├── hybrid_search.py vecteur + BM25 + RRF + Reranking
    ├── indexer.py       Docling → chunks → pgvector
    └── memory.py        Mem0 — mémoire cross-sessions

```

21.1 RAG Hybride — +30-40% de pertinence

```

async def hybrid_search(query: str, user_ctx: UserContext):
    embedding = await embed(query)

    # ABAC filter appliqué AVANT la recherche
    abac_filter = {
        'departement_id': user_ctx.dept_id,
        'visible_roles': {'$in': user_ctx.roles}
    }

    # Dual search en parallèle
    vector_results, keyword_results = await asyncio.gather(
        pgvector.query(embedding, filter=abac_filter, limit=20),
        pg_fulltext.search(query, filter=abac_filter, limit=20)
    )

    # Fusion — Reciprocal Rank Fusion
    merged = reciprocal_rank_fusion(vector_results, keyword_results, k=60)

    # Cross-encoder reranking — 5 résultats ultra-pertinents
    return await cohere_rerank(query=query, documents=merged, top_n=5)

```

21.2 Config Agent — Instructor + Pydantic

```

class ERPConfig(BaseModel):
    metiers: List[str]          # UNIQUEMENT dans METIERS_CATALOGUE
    modules: List[Module]       # héritage + overrides
    roles: List[Role]           # RBAC + attributs ABAC
    workflows: List[Workflow]   # validés comme DAG
    abac_rules: List[ABACRule]

```

```
    pipelines: List[Pipeline] # ActionEngine pipelines
    ux_profile: str           # UNIQUEMENT dans UX_PROFILES

# Instructor garantit le schema – impossible de sortir du catalogue
# Si JSON invalide → relance automatique jusqu'à 3 fois
# Si toujours invalide → questions de clarification au client
```

21.3 Mémoire utilisateur — Mem0

```
# Mem0 extrait les faits importants après chaque interaction :
# 'L'utilisateur cherche souvent les commandes du mois'
# 'Il préfère les exports en XLS'
# 'Il travaille principalement sur le département RH'

# Avant chaque appel LLM :
memories = await memory.search(query=query, user_id=user_id, limit=5)
# → injecté dans le system prompt pour personnalisation
```

PARTIE 10 — Algorithmique Avancé — Ce que les Autres n'ont Pas

24. Algorithmes Différenciants

La plupart des ERP IA s'arrêtent au RAG basique. Ces 5 algorithmes constituent l'avantage technique défendable qui prend des années à construire — et encore plus à maîtriser.

22.1 DAG — Validation des Workflows

Directed Acyclic Graph — Niveau Intermédiaire-Avancé

Pourquoi : Un workflow ERP est un graphe orienté (Commande → Stock → Validation → Livraison).

Ce qu'il faut maîtriser : Topological Sort (Kahn's algorithm), détection de cycles, BFS/DFS.

Application : Valider qu'un workflow généré par l'IA est cohérent — pas de cycles, toutes les étapes atteignables.

Bibliothèque : graphlib (JS), networkx (Python)

22.2 CRDT — Synchronisation Offline

Conflict-free Replicated Data Types — Niveau Avancé

Pourquoi : Quand un livreur modifie une livraison offline et que le bureau modifie la même livraison online, qui gagne ?

Ce qu'il faut maîtriser : LWW-Register, Vector Clocks, Operational Transform.

Application : Merge automatique sans conflit ni perte de données.

Alternative pour commencer : Timestamp + server-wins avec conflict queue pour les cas ambigus.

22.3 Rete Algorithm — Règles ABAC Performantes

Rete Algorithm — Niveau Avancé

Pourquoi : Évaluer des centaines de règles ABAC pour des milliers d'utilisateurs naïvement est $O(n \times m)$ — trop lent.

Rete précalcule un réseau de patterns et réduit drastiquement les évaluations.

Application : 'Cet utilisateur peut-il approuver cette commande ?' évalué en $O(1)$ après compilation.

Bibliothèque : Nools (JS), Drools (Java), ou implémentation custom NestJS

22.4 FSM — États ERP (XState)

Finite State Machine — Niveau Intermédiaire

Pourquoi : Une commande ERP passe par des états stricts (brouillon → soumise → validée → livrée → facturée).

Une FSM garantit qu'une transition illégale est impossible — tu ne peux pas facturer une commande non livrée.

Application : Générer automatiquement les FSM depuis la config conversationnelle. L'IA décrit, XState modélise.

Bibliothèque : XState (NestJS), statemachine (Python)

22.5 Hybrid RAG — Pertinence Maximale

Hybrid Search + Cross-Encoder Reranking — Niveau Avancé

Pourquoi : RAG basique (embedding seul) rate 30-40% des résultats pertinents.

Hybrid Search = vecteur (sémantique) + BM25 (keyword) fusionnés par Reciprocal Rank Fusion.

Cross-Encoder Reranking = trie les 20 meilleurs résultats avec un modèle plus précis.

Stack : pgvector + pg_trgm + Cohere Rerank — tout dans PostgreSQL, pas de DB externe.

PARTIE 11 — Compétences & Feuille de Route

25. Compétences à Maîtriser

23.1 Flutter & BDUI

Compétence	Niveau	Pourquoi
Dart avancé (async, streams, isolates)	Expert	BDUI + SSE streaming + engines concurrents
State management (Riverpod/Bloc)	Expert	7 engines + état complexe cross-widgets
Material 3 + ThemeData custom	Avancé	Design System natif — tokens, composants, adaptive
Flutter Multi-plateforme (mobile/web/desktop)	Avancé	Un seul codebase — 3 targets
Adaptive layouts	Avancé	UI adaptée mobile/desktop/web sans duplication
BDUI Engine (JSON → Widgets)	Expert	ComponentRegistry, RuleEvaluator, LayoutResolver
ActionEngine + pipelines	Expert	Orchestration des 7 engines
FormEngine (validation + calculs live)	Expert	Saisie métier sans if dans le code
Drift / Isar (SQLite)	Avancé	Offline-first + CRDT sync queue
CapabilityRegistry (natif)	Avancé	Scanner, GPS, print, NFC, biométrie — platform channels
Widgetbook	Intermédiaire	Maquettage vivant par composants + états
WebSockets + SSE	Avancé	Streaming réponses IA token par token

23.2 NestJS & Backend

Compétence	Niveau	Pourquoi
NestJS modules/guards/interceptors	Expert	Architecture ERP + 6 couches sécurité
AlgoEngine TypeScript	Expert	Fonctions typées + évaluation JSON
CASL + Casbin	Avancé	ABAC complexe multi-attributs
JWT + OAuth2 (Passport)	Avancé	Auth multi-tenant
Prisma / TypeORM	Avancé	Schema-per-tenant + migrations

Bull/BullMQ (job queues)	Avancé	Tâches async, sync offline queue
MinIO (@aws-sdk/client-s3)	Intermédiaire	Storage fichiers S3-compatible
WebSocket Gateway (SSE)	Avancé	Streaming IA + notifications temps réel

23.3 IA & Data Layer

Compétence	Niveau	Pourquoi
LangChain Agents	Avancé	Config Agent + orchestration LLM
Instructor + Pydantic	Expert	Extraction structurée fiable, anti-hallucination
pgvector + embeddings	Avancé	RAG vectoriel filtré par ABAC
Hybrid Search (RRF + Reranking)	Avancé	+30-40% pertinence vs RAG basique
Mem0	Intermédiaire	Mémoire utilisateur cross-sessions
Docling (IBM)	Intermédiaire	Parsing PDF, Excel, Word → chunks
Prompt engineering	Expert	Config Agent + extraction métier fiable
Streaming SSE FastAPI	Avancé	Token par token vers Flutter
LLM principal	DeepSeek V4-Flash (hébergé)	Sur ton cluster GPU — mutualisé tous tenants
LLM raisonnement	DeepSeek V4-Pro (hébergé)	Config Agent, tâches complexes d'agents
LLM fallback	Claude API (Sonnet)	Données critiques — paie, finance, santé
Cluster GPU	RTX 4090 × 2-4 → A100 à l'échelle	DeepSeek V4 distillé hébergé par toi

23.4 Algorithmique

Algorithme	Niveau	Application dans l'ERP
DAG + Topological Sort	Intermédiaire-Avancé	Validation workflows générés par l'IA
CRDT (Vector Clocks)	Avancé	Sync offline sans conflit ni perte
Rete Algorithm	Avancé	ABAC O(1) pour milliers d'utilisateurs
FSM (XState)	Intermédiaire	États ERP — transitions illégales impossibles

Reciprocal Rank Fusion	Intermédiaire	Fusion vecteur + BM25 pour RAG hybride
Property-based testing	Intermédiaire	Tests AlgoEngine (fast_check, glados)

26. Feuille de Route — 3 Phases

26. Feuille de Route Technique

La roadmap technique est alignée avec la vision 3 phases de Scalario. Phase 1 livre de la valeur client immédiatement. Phase 2 affine le moteur. Phase 3 scale.

Phase 1 — Fondations Scalario (Mois 1-3)

Objectif : livrer les premiers ERP à des clients réels. Scalario Labs opère Scalario Forge manuellement.

- JSON Schema BDUI avec variant — contrat TypeScript/Dart validé (Scalario Canvas)
- NestJS : auth JWT, RBAC Guards, structure multi-tenant + @nestjs/swagger (Scalario Shield)
- Flutter : Scalario Canvas + variantes + SyncStatusBar + NavigationConfig + Widgetbook
- i18n : ARB files FR/EN + lint no_hardcoded_strings dès le premier commit
- PostgreSQL : schema-per-tenant, RLS, pgvector, tenant_configs (Scalario Vault)
- Redis : sessions, cache config
- CASL : ABAC basique — Scalario Shield opérationnel
- Scalario Calc : ~30 fonctions atomiques, typage Zod strict
- DAG : validation des workflows — Scalario Flow de base
- Scalario Live : WebSocket in-app + FCM/APN enregistrement
- Login flow : Scalario → login → config chargée → UI rendue selon rôle

Phase 2 — Engines & IA (Mois 4-6)

Objectif : automatiser la config avec Scalario Forge. Premiers clients configurés en self-service supervisé.

- Scalario Form : validation, calculs temps réel, capacités dans champs
- Scalario Sense : scanner, camera, signature, imprimante, GPS
- Mobile Money : Wave + Orange Money + MTN MoMo + webhooks NestJS (Scalario Sense)
- Scalario Live : FCM/APN push réel + ConflictReviewScreen + SyncStatusBar actif
- Scalario Sync : CRDT Vector Clocks — résolution auto + ConflictReviewScreen
- Scalario Flow complet : pipelines JSON, variables, on_error strategies
- Scalario Vault niveau 3 : catalogue SQL par métier
- Scalario Mind : DeepSeek V4 hébergé, RAG hybride, SSE, Mem0, Docling
- Scalario Forge : Business Profile → Instructor → ERPConfig automatique
- Scalario Stage : sandbox multi-rôles, données fictives, ajustements live
- i18n Phase 2 : ARB Bambara, Wolof, Dioula — langues africaines

Phase 3 — Robustesse & Scale (Mois 7-12)

Objectif : moteur stable, scalable, prêt pour Phase 2 business (semi self-service).

- Scalaro Shield : Rete Algorithm — règles ABAC O(1) pour milliers d'users
- FSM XState : états ERP générés automatiquement par Scalaro Forge
- Scalaro Watch : Langfuse + anti-hallucination 6 couches opérationnelles
- Scalaro Mind : upgrade cluster GPU DeepSeek V4 selon volume clients
- Performance : memoization Scalaro Calc, vues matérialisées PostgreSQL
- i18n Phase 3 : Haoussa, Arabe — extension Maghreb + Sahel
- Multi-région : réplication DB pour clients dans plusieurs pays africains
- Swagger public : documentation API pour intégrateurs Phase 3
- Préparation Phase 2 business : portail client, billing Mobile Money, self-service partiel

26b. Scalario Network — Vision Phase 4

Scalario Network est la couche inter-tenants. Elle transforme Scalario d'un outil de gestion interne en un réseau d'échanges B2B natif entre entreprises. C'est une vision long terme — les décisions d'architecture actuelles ne doivent pas la bloquer.

26b.1 Ce que Scalario Network permet

```
Aujourd'hui – tenants isolés :
  Pharmacie Kosyam [tenant A] → gestion interne
  Grossiste Pharma [tenant B] → gestion interne
  Aucune interaction native entre les 2

Phase 4 – Scalario Network :
  Pharmacie Kosyam [tenant A]
    → voit le catalogue de Grossiste Pharma
    → passe une commande depuis son Scalario
    → paye via Mobile Money natif
    → reçoit le bon de livraison dans son Scalario

  Grossiste Pharma [tenant B]
    → reçoit la commande nativement
    → la traite dans son workflow Scalario
    → envoie la livraison + facture cross-tenant
    → est réglé automatiquement

  Tout tracé, tout audité, sans quitter Scalario.
```

26b.2 Les 4 niveaux de la plateforme

Niveau	Description
Niveau 1 — User	Utilisateur dans son tenant (vendeur, manager, DG...)
Niveau 2 — Tenant	Entreprise cliente de Scalario (Pharmacie Kosyam)
Niveau 3 — Scalario Platform	Infrastructure hébergée par Scalario Labs
Niveau 4 — Scalario Network	Réseau inter-tenants — B2B, marketplace, transactions

26b.3 Cas d'usage Scalario Network

Cas	Description	Qui bénéficie
Commerce B2B	Grossiste publie catalogue → détaillant commande nativement	Grossistes + détaillants

Sous-traitance	Chef chantier trouve fournisseur matériaux sur le réseau	BTP + fournisseurs
Marketplace sectorielle	Toutes les pharmacies en gros → détaillants comparent et commandent	Secteur pharmaceutique
Règlement inter-tenant	Mobile Money natif entre les 2 tenants — tracé et audité	Tous les tenants
Catalogue réseau	Un tenant publie produits/services visibles par le réseau	Fournisseurs
Historique cross-tenant	Bon de commande visible des 2 côtés — réconciliation auto	Acheteurs + vendeurs

26b.4 Décisions d'architecture dès Phase 1

Scalario Network n'est pas implémenté en Phase 1. Mais ces décisions prises maintenant évitent de tout reconstruire en Phase 4.

```
-- Ajouter dès Phase 1 dans public.tenants
ALTER TABLE public.tenants ADD COLUMN
  handle      TEXT UNIQUE,      -- '@grossiste-pharma-ouaga'
  network_public BOOLEAN DEFAULT FALSE,
  network_profile JSONB DEFAULT '{}';

-- Config tenant – network désactivé par défaut
{
  "network": {
    "public": false,
    "expose_modules": [],
    "allow_inbound_orders": false,
    "allow_inbound_payments": false
  }
}

-- Impact Phase 1-3 : zéro
-- Impact Phase 4   : infrastructure prête
```

26b.5 Ce que Scalario Network ajoute en Phase 4

Nouveaux composants Phase 4

Scalario Network — identité publique + catalogue réseau + découverte de partenaires.
 Scalario Trade — moteur de transaction cross-tenant (commandes, devis, factures B2B).

Scalario Market — marketplace sectorielle par verticale (pharma, BTP, restauration...).

API inter-tenant avec permissions granulaires — un tenant contrôle ce qu'il expose.

KYC des entreprises sur le réseau — notation, historique, contrats numériques.

Mobile Money cross-tenant — règlement automatique entre les 2 entreprises.

Ce que l'architecture actuelle protège pour Phase 4

tenant_handle : identité unique sur le réseau — à ajouter dès Phase 1.

schema-per-tenant : isolation parfaite — les données cross-tenant passent par une API dédiée, jamais par la DB directement.

Scalario Shield : RBAC/ABAC s'étend naturellement aux permissions inter-tenant.

Mobile Money dans Scalario Sense : déjà là — le règlement cross-tenant s'appuie dessus.

Scalario Live : les events cross-tenant (commande reçue d'un autre tenant) passent par le même mécanisme.

27. Résumé Final — Ce qui Différencie

SAP / Odoo / ERP classique	Ton ERP SaaS IA-Driven
Conçu pour grandes entreprises avec DSI	Conçu pour PME africaines sans DSI
Config : consultant + 3-6 mois	Config : Business Profile + IA + 45 min
Installation serveur côté client	SaaS — tu héberges, le client accède
APK différente par client ou absente	1 Scalario universel — login → UI générée
UI fixe imposée à tous	BDUI — UI selon tenant + rôle + contexte + variantes
RBAC simple	RBAC + ABAC + RLS — 5 couches
Offline basique ou inexistant	Offline CRDT — sync intelligente par diff
RAG absent ou basique	Hybrid RAG + DeepSeek V4 hébergé
Modules codés en dur	7 engines + JSON — zéro code par client
Prix inaccessible pour PME	Abonnement mensuel accessible
Pas localisé pour l'Afrique	XOF, Mobile Money, langues africaines, zones sans réseau
Chaque entreprise isolée	Scalario Network — B2B natif entre tenants en Phase 4

Principe directeur

Scalario Labs livre d'abord. Scalario scale ensuite. Scalario Network connecte enfin.

Scalario Forge configure en 45 min ce qui prenait 6 mois.

7 engines communicants — Canvas, Form, Calc, Sense, Vault, Live orchestrés par Flow.

Scalario Live = le seul engine déclenché par le serveur — notifications, refresh, badges en temps réel.

Scalario Mind (DeepSeek V4 hébergé) + Scalario Shield (RBAC/ABAC/RLS) = moteur sécurisé.

1 Scalario universel. Login → config chargée → UI générée selon rôle.

Phase 4 : Scalario Network transforme chaque tenant en nœud d'un réseau B2B africain natif.

C'est l'intégration cohérente de tout ça qui est défendable — pas une technologie seule.

— Fin du document — Scalario v12.0 — Scalario Labs — Mai 2026 —